

SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Generalizing Tracking Models under Sim-to-Real Domain-Shift for Guidewire Tracking in X-ray Fluoroscopy

Oğuz Kaan İmamoğlu



SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Generalizing Tracking Models under Sim-to-Real Domain-Shift for Guidewire Tracking in X-ray Fluoroscopy

Verallgemeinerung von Tracking-Modellen unter Sim-to-Real-Domain-Shift für das Guidewire-Tracking in der Röntgen-Fluoroskopie

Author:	Oğuz Kaan İmamoğlu
Supervisor:	Prof. Dr. Nassir Navab
Advisor:	Baochang Zhang
Submission Date:	29.07.2025

I confirm that this master's thesis is my own work and I have documented all sources and material used.

Munich, 29.07.2025 Oğuz Kaan İmamoğlu

Acknowledgments

I would like to first and foremost thank my advisor, Baochang Zhang, for guiding and supporting me throughout this challenging process. I would also like to express my gratitude to my supervisor Prof. Dr. Nassir Navab, for whom I feel very proud to have had the opportunity to work in his chair. I would also like to thank my family, especially my mother Ayşe and my sister Gül, for being by my side during times of stress and uncertainty, and for their continuous support throughout this journey.

Abstract

Guidewire tracking plays an important role in minimally invasive vascular interventions, where clinicians rely on real-time X-ray fluoroscopy to navigate guidewires through complex and narrow vascular structures. Accurate and reliable guidewire localization is essential for the safety and success of procedures such as angioplasty, stent placement, and catheterization. However, the fact that guidewires are not always clearly visible in X-ray fluoroscopy and the difficulty of accessing real-world annotated datasets make this automated tracking task challenging.

This thesis presents a simulation-based approach for training deep learning models to detect and track guidewires in X-ray fluoroscopy. The study addresses the challenge of limited annotated fluoroscopic data by generating realistic Digitally Reconstructed Radiographs (DRRs) from Computed Tomography Angiography (CTA) volumes. These DRR sequences are enhanced with simulated guidewire movement and temporally varying deformation fields to simulate cardiac motion, enabling the creation of anatomically realistic training sequences without the need for manual annotation.

To bridge the domain gap between synthetic and real X-ray images, a pseudo-labeling–based domain adaptation strategy is proposed. A YOLOv8 model trained on DRRs is iteratively fine-tuned using high-confidence predictions on unlabeled real X-ray data. This unsupervised domain adaptation method allows the model to gradually align with the visual characteristics of real fluoroscopy sequences. In addition, the QDTrack model is used to assess how other tracking approaches perform for this task, offering a comparison between spatial and sequence-based tracking models.

Experiments on real clinical datasets show that the proposed pipeline achieves competitive performance without relying on annotated real images. The results show that simulation-based data, when used with appropriate domain adaptation strategies, can offer a scalable and solution for guidewire tracking. This approach holds promise for the broader development of learning-based tools in interventional imaging, particularly in scenarios where acquiring labeled data is challenging or impractical.

Contents

Acknowledgments **iii**

Abstract **iv**

1 Introduction **1**
1.1 Motivation 1
1.2 Problem Definition 3
1.3 Research Questions and Contributions 4
1.4 Thesis Outline 5

2 Background and Related Work **7**
2.1 Key Concepts and Terminology 7
2.1.1 Computed Tomography Angiography (CTA) 7
2.1.2 X-ray Fluoroscopy 8
2.1.3 Guidewire 8
2.1.4 Digitally Reconstructed Radiographs (DRR) 8
2.2 DRR Generation Frameworks 9
2.2.1 DeepDRR 9
2.2.2 DiffDeepDRR 9
2.3 Detection and Tracking Models 10
2.4 Related Works 10
2.4.1 Domain Adaptation Between DRR and Real X-ray 11
2.4.2 Deep Learning for Guidewire Tracking 11

3 Methodology **13**
3.1 Data Acquisition and Preprocessing 14
3.1.1 Datasets 14
3.1.2 Data Preprocessing 16
3.2 Registration and Deformation Field Extraction 23
3.2.1 Rigid Registration 24
3.2.2 B-Spline Registration 24
3.2.3 Usage of Registration in This Study 25

3.3 DRR Dataset Generation . . . 27
3.3.1 Simulation of Guidewire . . . 27
3.3.2 Applying Motion of Heart . . . 28
3.3.3 Generating DRR Sequences . . . 28
3.3.4 Obtaining DRR Dataset and Resampling . . . 31
3.4 Obtaining Guidewire Tracking Models . . . 32
3.4.1 YOLOv8 . . . 32
3.4.2 QDTrack . . . 38
3.5 Unsupervised Domain Adaptive Detection Pipeline . . . 42
3.5.1 Domain Augmentation Methods . . . 43
3.5.2 Choosing Real X-ray Dataset for Pseudo Labeling . . . 45
3.5.3 Pseudo Labeling Strategy . . . 45
3.5.4 Choosing the Best Iteration . . . 46

4 Experiments and Results **48**
4.1 Evaluation Metrics and Tools . . . 48
4.2 Evaluated Models . . . 50
4.3 Results on Simulation Dataset . . . 51
4.4 Ablation Study on Domain Adaptation Pipeline . . . 51
4.5 Results on Single Guidewire Real X-Ray Dataset . . . 52
4.6 Results on Double Guidewire Real X-Ray Dataset . . . 54
4.7 Implementation Details . . . 55

5 Discussion **56**
5.1 Discussion on Simulation Pipeline . . . 56
5.2 Discussion on Tracking Results . . . 57
5.3 Further Improvements . . . 58

6 Conclusion **59**

Abbreviations **61**

List of Figures **62**

List of Tables **63**

Bibliography **64**

1 Introduction

1.1 Motivation

Cardiovascular diseases are the leading mortality reason globally, accounting for an estimated 17.9 million deaths annually [1]. Europe also has a similar situation in this regard, with approximately 4 million deaths per year [2]. This severe situation still necessitates innovative approaches for early detection, accurate diagnosis and effective treatment of cardiovascular diseases. Under these circumstances the improvement in the medical technologies, especially imaging technologies are crucial for handling this widespread health problem.

X-ray fluoroscopy is an important part of modern image-guided vascular interventions which enables real-time visualization of medical devices inside the patient's body. This imaging technology is often used in minimally invasive treatments, to help physicians guide tools such as guidewires and catheters through complex vascular systems. Among these tools, guidewires are particularly important, since they are the first tool that gets to the patient's body and serve as the navigational element for therapeutic devices [3, 4]. The ability to track guidewires accurately and reliably during a procedure is essential not only for clinical success but also for reducing procedural risks and improving patient outcomes.

However, this is a challenging task that requires both fine tracking ability and technological solutions. One complication in precise guidewire tracking is the excessive background noise caused by various overlapping textures, which makes the guidewire indistinguishable due to the low signal-to-noise ratio of X-ray images. Another difficulty encountered during the process is that the edges of long, thin organic structures, most notably vertebrae and ribs, often exhibit a wire-like appearance, which leads to confusion in tracking [4]. Studies show that as the force required by the physician to track the catheter or guidewire increases, the risk of harm to the patient during critical interventions also increases [5].

Technological improvements that will be developed for this purpose to assist the physician will both reduce the complications that the patient may encounter and improve the physical fatigue that the operator will experience. In recent years, the integration of artificial intelligence, particularly deep learning, into fluoroscopy-based systems has opened new possibilities for real-time navigation and decision support [6].

With their powerful data analysis capabilities, these systems significantly support interpretation by analyzing continuous X-ray image streams. However, training deep learning models requires a large volume of datasets and acquiring such datasets presents significant challenges [7]. First of all, due to ethical concerns and patient rights, obtaining clinical data is a difficult task. Secondly, due to their thin, flexible structure, guidewires are often not clearly visible in low-dose fluoroscopic images. Their appearance varies with pose, viewing angle, and occlusion, making manual annotation difficult and error prone [8]. Finally, it would be a very time-consuming process for a physician to watch and label the images one by one. As a result, publicly available datasets with pixel-level or even bounding-box annotations for guidewires are extremely rare.

Researchers have increasingly explored synthetic data generation as an alternative to cope with this limited resource situation. Computed Tomography Angiography (CTA), a medical imaging technique that provides detailed volumetric representations of the vascular system, is one of the common application areas in clinical studies where synthetic data is used [9]. Utilizing these static 3-dimensional data, it is possible to simulate how anatomical structures will appear under X-ray imaging conditions. Digitally Reconstructed Radiographs (DRRs) are synthetic X-ray-like images that are generated by projecting a 3-dimensional volume, such as CTA, onto a 2-dimensional plane using physically based rendering techniques [10]. These images mimic the appearance of real fluoroscopic scans and can be produced with high control over imaging parameters, projection angles, and anatomical detail. DRRs offer a flexible and scalable way to create labeled datasets for tasks such as guidewire detection and tracking, without the need for manual annotation or radiation exposure. Since DRRs can simulate realistic clinical situations, they're a valuable tool for training and testing deep learning models in interventional imaging.

Therefore, this work aims to explore how simulation-based data generation can be utilized to train deep learning models that enable accurate and robust guidewire tracking. In particular, it investigates which methods can be employed to bridge the domain gap between simulated and real-world data, focusing on strategies that enhance model generalization across domains. By relying entirely on synthetic data for training, this study seeks to eliminate the dependency on annotated real-world datasets, which are often scarce, time-consuming, and expensive to produce in medical contexts. The ultimate goal is to demonstrate that models trained exclusively on simulation-generated sequences can be effectively applied in real-world scenarios without additional supervision. Through this effort, the applicability of simulated data for downstream tasks, specifically guidewire tracking, is validated, offering a promising path toward safer and more efficient image-guided interventions.

1.2 Problem Definition

The primary problem addressed in this thesis is the significant lack of annotated data required to train deep learning models for guidewire tracking in fluoroscopic sequences. As mentioned previously, in clinical practice, obtaining large-scale datasets in which guidewire positions are accurately labeled over time is quite difficult for various reasons. As a result, existing datasets remain limited in both quantity and quality, often restricting model development to narrow, case-specific scenarios.

To cope with these data limitations, synthetic data generation has become a popular solution. However, this approach still accommodates a significant challenge due to the domain gap between synthetic and real fluoroscopic images. Models trained only on synthetic data often perform poorly when applied to real-world cases [11]. The main reason for this problem arises from differences in appearance, texture, noise characteristics, anatomical variation, and imaging artifacts of synthetic and real data. This discrepancy, commonly referred to as sim-to-real domain shift, prevents the direct application of simulated training data without additional adaptation or fine-tuning [12].

From this standpoint, this work aims to explore how simulation-based data generation can be leveraged to train deep learning models that enable accurate and robust guidewire tracking. In particular, the methods that can be employed to bridge the domain gap between simulated and real-world data are examined. Utilizing synthetic data for training deep learning models, the dependency on annotated real-world datasets, which are often scarce, time-consuming, and expensive to produce in a clinical context, can be removed. The overarching aim is to demonstrate that models trained exclusively on simulation-generated sequences can be effectively applied in real-world scenarios without additional supervision. By this means, the applicability of simulated data for guidewire tracking is validated. A potential avenue for safer and more efficient image-guided treatments is opened up through this effort.

By showing that DRR-based data enables effective model training and generalization to real X-ray sequences, this work contributes to the broader understanding of DRRs as a bridge between preoperative imaging and real-time procedural data. Ultimately, the intention is to validate DRRs not only as a task-specific workaround but as a general-purpose synthetic data strategy that minimizes the domain gap and accelerates the development of robust, clinically applicable AI systems.

1.3 Research Questions and Contributions

The primary goal of this thesis is to investigate whether synthetic fluoroscopic sequences generated from CTA data can effectively train deep learning models for the task of guidewire tracking. This objective leads to several research questions that guide the scope and direction of the work:

Research Questions

- Can simulated DRR sequences generated from CTA data effectively train deep learning models for guidewire tracking?
- Does applying deformation fields to model cardiac and guidewire motion lead to realistic and meaningful simulation data?
- How well can models trained solely on simulated data generalize to real X-ray fluoroscopy sequences, despite the domain shift?
- What strategies can be used to reduce the domain gap between simulated and real X-ray data and improve detection and tracking performance?
- How can state-of-the-art detection and tracking models be optimized for guidewire tracking in X-ray fluoroscopy scenarios?

Contributions

To answer the above questions, this thesis makes the following key contributions:

- A comprehensive simulation pipeline is developed to generate dynamic DRR sequences from CTA data. This includes the synthesis of guidewire trajectories and the application of temporally varying deformation fields that simulate heart motion and guidewire movement in the DRR sequences.
- Detection and tracking models are trained with this simulation data and their performance is measured on real-world datasets.
- An unsupervised domain adaptation strategy is developed to reduce the domain gap between simulation and real data, and the impact of this strategy on tracking performance and generalization capability is evaluated.
- The resulting tracking models are tuned to be successful in guidewire tracking scenarios.

This study explores the use of synthetic data for guidewire tracking by generating realistic DRR sequences, with simulated guidewire movement and heart motion. Tracking models are trained on this data and evaluated on real X-ray sequences. To improve generalization and reduce the domain gap between simulated and real world data, an unsupervised domain adaptation method is proposed. The results show that simulation-based training, combined with domain alignment, can effectively support guidewire detection and tracking without relying on manual annotations.

1.4 Thesis Outline

This thesis is organized into six chapters, each addressing a specific aspect of the research:

- **Chapter 1 – Introduction:**
  Introduces the topic and explains the motivation behind the study. It defines the research problem, states the objectives, presents the research questions, and summarizes the contributions. It also provides an overview of the thesis structure.
- **Chapter 2 – Background and Related Work:**
  Reviews the fundamental concepts of medical imaging, with a focus on CTA, X-ray fluoroscopy, and DRR generation. It discusses relevant studies with this study.
- **Chapter 3 – Methodology:**
  Describes the proposed simulation pipeline in detail. It includes data preprocessing, extraction and application of deformation fields, synthetic guidewire generation, and DRR rendering process. The chapter also covers used detection and tracking models in the study, in addition to applied unsupervised domain adaptation strategy.
- **Chapter 4 – Experiments and Results:**
  Presents the experimental setup, including evaluation metrics, and data splits. It reports both quantitative and qualitative results from training and testing the guidewire tracking model on synthetic and real datasets.
- **Chapter 5 – Discussion:**
  This chapter evaluates and compares the results and tracking performance of the models. It also suggests further contributions to this study, including actions that can be taken to improve the realism of the simulated data or alternative domain alignment strategies that can be implemented.

- **Chapter 6 – Conclusion:**
  This final chapter summarizes the key findings of the thesis, emphasizing the potential of simulation-based training for guidewire tracking without requiring manually annotated data and effectiveness of domain adaptation strategies. It summarizes the main contributions of the study.

2 Background and Related Work

This chapter provides a detailed literature review to give readers insight into the current works and preliminary background information regarding the topic. By this means, the research gap is pointed out, and the contributions of this thesis stand out. The first section introduces the fundamental imaging techniques used in vascular interventions, including CTA and X-ray fluoroscopy, along with detailed explanations of a guidewire and DRRs. Following, DRR generation frameworks that used for this study are explained. In the third section, the fundamental detection and tracking algorithms used in this study are introduced. Finally, in the fourth section, related studies for the handled topic are provided. Through this review, the chapter highlights current limitations in the field and identifies the specific research gap that this thesis aims to address.

2.1 Key Concepts and Terminology

Modern interventional radiology relies heavily on advanced imaging technologies to visualize anatomical structures and medical tools during minimally invasive procedures. To understand the simulation pipeline proposed in this thesis, it is important to first review the fundamental modalities and clinical components involved. This section introduces key concepts such as X-ray fluoroscopy, CTA, and guidewire. All of them have crucial roles in image-guided vascular interventions. Additionally, the concept of DRRs, which forms the basis for simulated data generation in this study, is explained.

2.1.1 Computed Tomography Angiography (CTA)

CTA is a non-invasive medical imaging technique that allows physicians to visualize vascular structures. During a CTA scan, a contrast agent is injected into a vein while a Computed Tomography (CT) scanner capturing detailed cross-sectional images. Then, these images are used for reconstructing 3D view of the vascular system. This helps physicians to detect conditions such as blockages, aneurysms, or other abnormalities in arteries and veins. CTA is widely used in cardiac, cerebral, and peripheral vascular imaging, and it plays a crucial role in preoperative planning for image-guided inter-

ventions. Because it combines speed, detail, and it is non-invasive, CTA has become a valuable tool in modern diagnostic workflows [9].

2.1.2 X-ray Fluoroscopy

Fluoroscopy, on the other hand, provides real-time imaging by continuously projecting X-rays through the body and capturing them on a detector. This results in a sequence of 2-dimensional grayscale images showing the dynamic motion of internal structures and surgical tools [13]. Fluoroscopy is a key tool in guiding catheters and guidewires through the vascular system. However, because of concerns about radiation exposure, especially during long procedures, fluoroscopy is usually performed with low-dose settings [14]. This can reduce image quality and makes it difficult to clearly visualize fine structures like guidewires. Fluoroscopic images may also suffer from noise, motion blur, and overlapping anatomy, which further complicates interpretation.

2.1.3 Guidewire

Guidewires are thin, flexible medical devices used to access and navigate through blood vessels during minimally invasive procedures. They serve as the first instrument to cross narrowed or blocked areas and guide the placement of other tools, such as balloons, catheters, or stents [15].

Their thin and metallic structure makes them difficult to visualize clearly in fluoroscopy. Often, only the guidewire tip appears as a faint, moving shadow in the image, while the rest of the body is invisible or blends into the background [4].

2.1.4 Digitally Reconstructed Radiographs (DRR)

DRR is a synthetic X-ray image generated by projecting a 3D medical volume, such as a CT or CTA scan, onto a 2D plane using a specified imaging geometry. DRRs simulate the physical process of X-ray image formation and are widely used for image registration, surgical planning, and simulation-based training [10]. In contrast to real fluoroscopy, which captures dynamic anatomical motion using ionizing radiation, DRRs can be produced without radiation exposure and with complete control over projection angles, contrast, and simulated materials. This makes them highly suitable for generating large-scale datasets, especially when annotated clinical data are scarce or difficult to obtain.

2.2 DRR Generation Frameworks

The generation of high quality DRRs constitutes the most critical stage in the simulation pipeline, as the quality and realism of the DRRs directly impact the learning and tracking performance in real domain. Therefore, this section is devoted to a explanation of the DRR generation frameworks employed in this study. In particular, DiffDeepDRR [16] a differentiable extension of the DeepDRR [10] framework is adopted. Since DiffDeepDRR builds directly upon DeepDRR's simulation principles, the use of DiffDeepDRR inherently implies reliance on DeepDRR's physically grounded X-ray image formation model. The two frameworks are described in the following subsections.

2.2.1 DeepDRR

DeepDRR is a modular, learning-friendly framework designed to generate synthetic X-ray images from 3D CT data with high realism and computational efficiency. It integrates PyCUDA [17] and PyTorch [18] for GPU acceleration and deep learning compatibility. The framework consists of four main components: material decomposition, forward projection, scatter estimation, and noise injection.

- Material decomposition: A neural network separates tissues like air, bone, and soft tissue in the CT volume, providing the foundation for accurate imaging.
- Physics-aware ray tracing: It uses this tissue map to simulate X-ray beams realistically, using GPU acceleration for speed.
- Scatter estimation: A second network predicts how X-rays scatter through tissues, bringing back subtle image artifacts often missed in simple projections.
- Noise injection: Finally, it adds realistic sensor noise to match real clinical conditions.

The result is a set of synthetic DRRs that look like real X-ray fluoroscopy images. Models trained with data generated by DeepDRR have been shown to perform impressively on actual clinical images, without needing any additional domain adaptation [10]. By offering an open-source, PyTorch-based implementation, it makes high-quality simulated data practical for deep learning research in medical imaging.

2.2.2 DiffDeepDRR

DiffDeepDRR is based on DeepDRR and offers some improvements. It offers faster and fully differentiable pipeline for DRR generation. This differentiability enables seamless

integration into deep learning workflows, allowing gradients to flow through the image synthesis process. As a result, parameters can be optimized end-to-end during training. Its ability to generate DRR images rapidly makes it a good fit for scenarios when a large volume of data is needed to be produced.

2.3 Detection and Tracking Models

This section introduces the main detection and tracking methods that are used in this study. These methods are chosen due to their strong performance in object detection and multi-object tracking tasks, as well as their applicability to medical imaging contexts. Here, only a brief introductory description is provided for each, while their specific roles within this study, how and why they are used, their strengths and limitations are discussed in detail in the methodology chapter.

The first model is YOLOv8 [19, 20], a real-time object detection algorithm from the well-known YOLO family [21]. It includes several improvements over earlier versions, such as a redesigned backbone and a decoupled head, which help increase both accuracy and speed. Its lightweight structure and fast inference also make it practical for use in real-time systems.

Since YOLOv8 is a detection algorithm, it must be paired with a tracking method to follow objects across consecutive frames. For this purpose, BoT-SORT [22] is used to build a complete tracker. BoT-SORT is based on SORT [23], a well-known and widely used tracking algorithm that offers a simple yet effective solution for many multi-object tracking tasks.

The final method considered is QDTrack, a tracking-by-detection approach that focuses on matching objects across frames based on their visual appearance [24]. In this study, QDTrack is used as a reference method to compare with the proposed YOLO-based trackers. Its strong performance in general tracking tasks makes it a suitable baseline for evaluation.

2.4 Related Works

The growing need for robust and generalizable models in fluoroscopic image analysis has led to increasing interest in synthetic data generation, domain adaptation, and image-guided tool tracking. This section reviews prior work across two main categories relevant to this thesis: approaches that aim to bridge the domain gap between DRR and real X-ray images, and deep learning-based methods for guidewire detection and tracking. These works collectively demonstrate the feasibility of synthetic data pipelines and inform the methodological choices made in this thesis.

2.4.1 Domain Adaptation Between DRR and Real X-ray

DRRs have been widely utilized as a surrogate for real X-ray images in deep learning pipelines, especially when annotated X-ray data is limited or unavailable. Y. Zhang, Miao, Mansi, and Liao [25] proposed a task-driven Generative Adversarial Network (GAN) for unsupervised domain adaptation from synthetic DRRs to real X-ray images. Their approach first trains a Dense Image-to-Image network on labeled DRRs generated from CT volumes for multi-organ segmentation. Subsequently, a modified cycle-GAN [26] architecture is employed to map real X-rays to the DRR domain, while maintaining segmentation performance through deep supervision. The method significantly improves segmentation accuracy on topogram images without requiring any labeled X-ray data, highlighting the potential of DRRs as effective training priors.

This work shares conceptual similarities with the current study, particularly in its reliance on DRRs to train image analysis models in the absence of annotated real X-ray data. However, there are two notable distinctions. While Y. Zhang, Miao, Mansi, and Liao [25] focus on multi-organ segmentation, this work targets guidewire detection and tracking. They also employ a cycle-GAN-based domain adaptation module, specifically a Task-Driven GAN architecture, to align DRRs with real fluoroscopic images. On the other hand, this work leverages a pseudo-labeling-based unsupervised domain adaptation strategy to refine models via high-confidence predictions on unlabeled real domain images.

2.4.2 Deep Learning for Guidewire Tracking

Accurate tracking of guidewires in X-ray fluoroscopy is essential for the success of minimally invasive procedures, where real-time navigation is required under challenging imaging conditions. Due to factors such as low radiation dose and motion artifacts, guidewires often appear faint and fragmented, making traditional image processing techniques unreliable. In recent years, deep learning-based methods have gained traction for their ability to learn complex patterns and achieve robust performance in such difficult settings. These approaches typically combine detection networks with temporal modeling or tracking components to localize and follow guidewires across frames, enabling more efficient and accurate support for interventional workflows.

B. Zhang, Bui, C. Wang, et al. [3] proposed a deep learning framework for real-time tracking and segmentation of guidewires in intraoperative X-ray fluoroscopy. The method consists of two main components: a modified YOLOv5 network for guidewire region detection, and a segmentation module named Hessian-Net. The first stage rapidly localizes the guidewire tip by predicting bounding boxes, while the second stage applies a combination of attention mechanisms and vessel-enhancing filters to

segment the guidewire tip in detail. This hierarchical approach allows for both coarse localization and fine-grained shape recovery in real time, addressing key challenges in fluoroscopy-guided procedures.

One of the main contributions of the work is the demonstration that, given access to sufficiently annotated real X-ray sequences, deep learning models can achieve high accuracy in guidewire detection. By training and validating their model on clinical data, the authors show that accurate and robust guidewire tracking and segmentation is feasible in real intraoperative settings. The use of YOLOv5 in this work shows that lightweight object detectors can handle guidewire detection tasks efficiently and fast enough for real-time use in clinical settings. Seeing its success in real data has also been a motivation for choosing a similar detection approach in this study.

In contrast, the current work focuses on training detection models entirely on simulated DRR datasets. No annotated real X-ray data is used during training, which introduces a domain gap between training and testing conditions. Different from to study of B. Zhang, Bui, C. Wang, et al. [3], the contribution of this study is to achieve high-level tracking performances by using simulation training datasets, without using annotated real-world datasets.

3 Methodology

This chapter describes the simulation pipeline developed to generate synthetic fluoroscopic image sequences and train deep learning models for guidewire detection and tracking. The main goal is to deal with the limited availability of annotated fluoroscopy data by creating realistic Digitally Reconstructed Radiographs (DRRs) from Computed Tomography Angiography (CTA) volumes. These DRRs are enhanced with synthetic guidewire paths and dynamic anatomical motion, making them useful for training and evaluating learning-based methods.

The pipeline is organized into several main steps: preprocessing of raw data, extracting deformation fields to simulate cardiac motion, generating guidewire trajectories, and rendering DRR sequences using DiffDeepDRR. At the end of this process, a labeled synthetic dataset is obtained, which can be used to train and test different tracking models.

In this work, a YOLOv8 is used to localize the guidewire tip position in each frame. Alongside this, QDTrack is also tested to study how well temporal information can be captured across image sequences. The two models are compared to better understand the strengths and limitations of different type of tracking models.

To make the synthetic data more compatible with real fluoroscopy, a domain adaptation pipeline is employed. This includes a pseudo-labeling strategy, where confident predictions on unlabeled real X-rays are used to gradually fine-tune the model. Although pseudo-labeling originates from semi-supervised learning, generating "pseudo-labels" from high-confidence model outputs, it has also become a widely accepted method within unsupervised domain adaptation to help align source and target domains [27, 28]. Even without manual annotations, this process helps the model adapt to the visual characteristics of real clinical data and effectively reduce the domain gap between synthetic and real datasets.

The rest of the chapter explains each part of this process in detail, with a focus on how the pipeline is designed to maintain anatomical accuracy and create a training setup that generalizes well to real-world images.

3.1 Data Acquisition and Preprocessing

3.1.1 Datasets

Having access to reliable and varied datasets is key when training and evaluating deep learning models for medical image analysis. In this study, several datasets are used across different stages of the simulation pipeline, as well as during model training and testing.

These datasets differ in terms of imaging modality, temporal features and clinical origin. The selected datasets enable the modeling of realistic anatomical motion, the generation of simulation image sequences, and the evaluation of model generalization to real-world clinical data. This section introduces these datasets that are used in this study, including temporal volumetric scans, static anatomical references, and real fluoroscopic sequences captured during clinical procedures.

4D CTA Dataset

The main dataset used in this study is a collection of four-dimensional Computed Tomography Angiography (4D CTA) scans obtained from six different patients. Each patient dataset consists of 20 separate 3D time phases, representing anatomical structures of the torso captured at different points during one complete cardiac cycle.

The dataset provides:

- The anatomical basis for generating simulation data,
- A reference to embed and simulate guidewire motion across time,
- Access to different anatomical states of the body across time-steps, which enables modeling temporal changes and generating motion in DRR sequences during simulation.

The temporal resolution of this dataset enables patient-specific simulations that reflect real cardiac dynamics.

ASOCA Dataset

The Automatic Segmentation of Coronary Arteries (ASOCA) dataset is a publicly available static CTA dataset commonly used in cardiovascular imaging research [29]. It consists of high-resolution CTA scans of the thoracic region, focused on coronary arteries and cardiac anatomy. It includes 3D CTA volumes and separate segmentation of the left and right coronary arteries from 40 patients: 20 diseased and 20 healthy. However, only data from healthy patients are used in this study.

While ASOCA lacks temporal dynamics, deformation fields obtained from the 4D CTA dataset are transferred and applied to its static volumes to create pseudo-temporal sequences. This allows DRR generation from ASOCA to mimic realistic cardiac motion, increasing anatomical diversity for training.

Real X-ray Fluoroscopy Datasets

To evaluate model generalization, real fluoroscopic image sequences acquired during PCI procedures are used. These datasets reflect authentic imaging conditions, including noise, occlusion, and tool variability.

Deutsches Herzzentrum (DHM) Dataset: This dataset contains fluoroscopic sequences from dual-guidewire coronary intervention cases. It captures real tool motion, breathing artifacts, and clinical complexity. It is used solely for testing generalization from synthetic training data to real clinical inputs. This dataset do not include any pre-existing annotations and is manually labeled during the study to enable quantitative evaluation of the tracking models. The labeling is performed using the *labelImg* [30] tool, and the resulting annotations are used solely as ground truth for computing performance metrics. This dataset, consisting of 10 sequences and 147 images, is not split and only be used as testing purposes. Throughout the rest of this work, this dataset is referred to as the *double guidewire real X-ray dataset*.

Single Guidewire Coronary Intervention Dataset: This is an in-house dataset collected from real coronary intervention procedures, containing high-quality X-ray fluoroscopy sequences with a single guidewire in view. It provides clean and temporally continuous frames, making it suitable for evaluating guidewire tracking models under realistic clinical conditions. Unlike other datasets, this one comes with pre-existing annotations. Throughout the rest of this work, this dataset is referred to as the *single guidewire real X-ray dataset*. This dataset, consisting of 20 video sequences and 5004 images, is split into training (60%), validation (15%), and test (25%) sets on a sequence basis. The training and validation subsets are used to train models on real X-ray data and serve as a upper bound reference for evaluating the effectiveness of proposed domain adaptation strategies. The test set is held out exclusively for assessing and comparing the performance of models trained on simulation against real X-ray data. The training split is also used for pseudo labeling process, which is described in following sections.

These datasets serve as critical evaluation sources to bridge the gap between synthetic simulations and real-world clinical scenarios.

3.1.2 Data Preprocessing

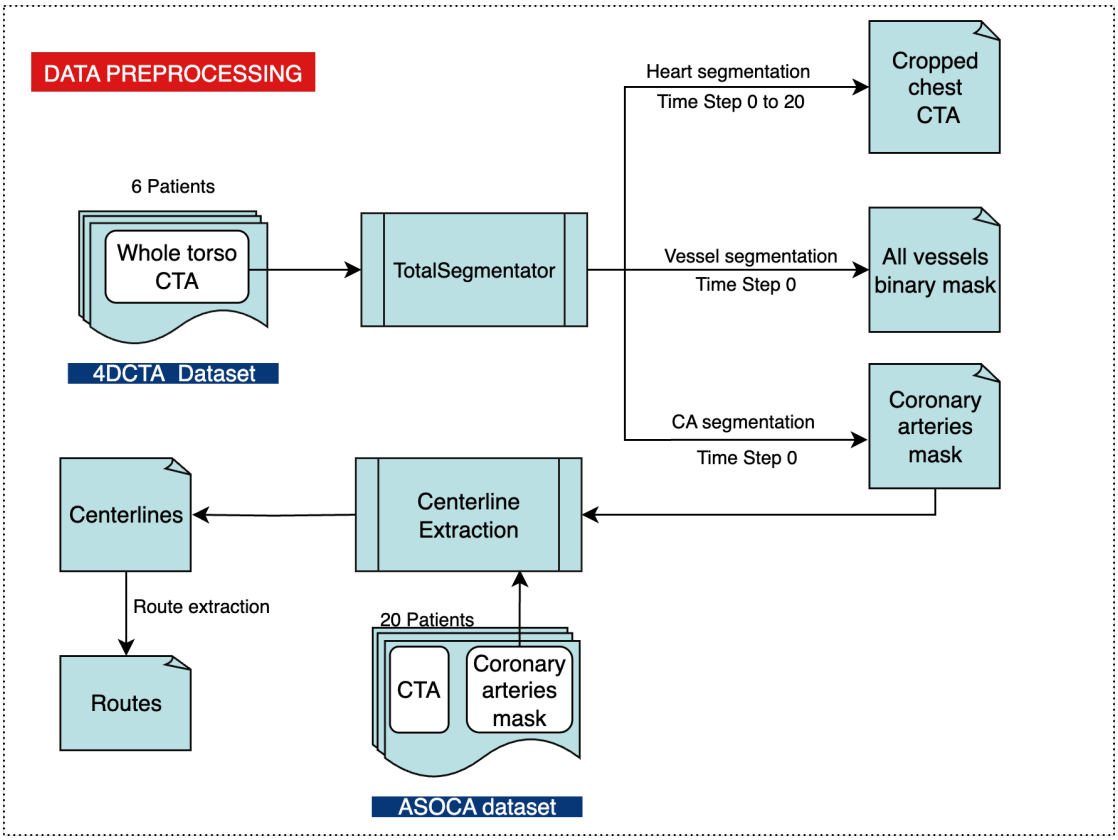


Figure 3.1: Overview of data preprocessing pipeline.

The goal of data preprocessing is to prepare the raw CTA volumes for simulation by focusing on the relevant anatomical region, reducing computational load, and ensuring anatomical accuracy. Since one of the primary objectives of this study is to simulate guidewire motion in the coronary artery region, preprocessing steps aim to remove unnecessary anatomical parts, define precise simulation zones, and enhance visual realism during DRR generation. To this end, segmentation, cropping, and vessel masking techniques are applied. These steps improve both the quality and efficiency of subsequent deformation modeling and DRR rendering, while ensuring that the resulting sequences are clinically meaningful and computationally manageable.

Heart Segmentation

Accurately identifying the heart region is an important first step in the preprocessing pipeline. It helps limit the simulation to relevant anatomical areas and reduces unnecessary computational load. Since this study focuses on guidewire navigation through the coronary arteries, the heart needs to be segmented precisely to define the region of interest for the rest of the workflow—such as cropping, vessel masking, and DRR generation.

For this purpose, a deep learning-based tool called *TotalSegmentator* [31] is used. The `heartchambers_highres` task is selected to generate a binary mask of the heart, which includes the cardiac chambers (atria and ventricles) also nearby structures like the myocardium and pericardium. This mask, saved in `Neuroimaging Informatics Technology Initiative (NIfTI)` format, serves as the anatomical reference for extracting the simulation region in later steps.

For the 4D CTA dataset, which contains dynamic cardiac scans across 20 time phases per patient, heart segmentation is performed on the first time phase ($t = 0$). It means, this segmentation process is applied only once per patient. The resulting mask is used to derive a 3D bounding box, which is then applied uniformly to all time phases of the same patient to ensure temporal consistency across the dynamic scan. The assumption is that cardiac motion remains spatially bounded within this extended region, which includes an additional fixed margin along all axes.

For the ASOCA dataset, which consists of static CTA scans already focused on the thoracic and cardiac region, no additional heart segmentation is required. The input volumes are pre-cropped and spatially normalized, and therefore considered ready for downstream analysis without further anatomical localization.

Data Cropping

After segmenting the heart, the next step is to crop the CTA volumes to focus on the relevant anatomical region and reduce unnecessary computational costs and ensuring time efficiency. This is especially important when working with time-series 3D data, as it helps speed up later steps like vessel segmentation, deformation modeling, and DRR generation without sacrificing important information.

In the 4D CTA dataset, cropping is done using the bounding box of the heart mask from the first time phase ($t = 0$). To make sure the heart and nearby structures are fully included across all time steps, a fixed margin is added on all sides of the bounding box. These margins are chosen to ensure a balance which keeps the region tight enough to remove unnecessary areas, but wide enough to include whole heart and coronary arteries to capture movement due to cardiac motion. Once calculated, the same crop

box is applied to all 20 phases of a patient to maintain consistent alignment over time. The cropped volumes are saved in `NIfTI` format. Each output is stored in a structured folder hierarchy based on patient ID and time-step.

No cropping is applied to the ASOCA dataset. These volumes are already focused on the heart and thoracic vessels, and their spatial size is small enough to avoid including irrelevant areas. Because of this, ASOCA scans are used as-is for vessel analysis and DRR simulation.

In terms of implementation, cropping is carried out using *SimpleITK* [32, 33, 34]. The code identifies the smallest and largest non-zero voxels in the segmentation mask, then expands them in each direction—making sure it stays within image boundaries.

On average, the cropping step reduces the CTA volume size from around 200MB to approximately 100MB, though the exact values may vary slightly between patients.

Orientation Normalization

When working with multiple datasets and multiple patients, it is crucial to ensure that all volume data shares the same anatomical orientation. Anatomical structures may appear flipped, rotated, or misaligned in the absence of consistent orientation, which can lead to issues with motion modeling, simulation, and visualization.

The scans in the 4D CTA dataset are obtained using different protocols and come from various patients. As a result, the image axes (left-right, front-back, and top-bottom) may have different orientations. All volumes are reoriented into the typical Left Posterior Superior (LPS) format, which is frequently utilized in medical imaging, in order to correct this [35]. In this manner, the same coordinate system is used for all patient scans. The `DICOMOrient` function from SimpleITK is used to perform the reorientation.

In contrast, the ASOCA dataset is acquired and preprocessed under a standardized imaging protocol. All volumes in this dataset are already aligned to the LPS coordinate convention, and therefore no additional reorientation is required.

Standardizing orientation across datasets is a prerequisite for any cross-domain alignment task. In this study, certain downstream processes such as registering 4D CTA volumes to ASOCA references rely on consistent orientation. Without this normalization step, registration algorithms may produce anatomically implausible transformations or fail to converge due to axis mismatches.

Table Removal

CTA scans can include parts of the patient support table, which often appear as high-density areas near the lower slices of the volume. These structures may cause

strong attenuation artifacts in DRR projections, leading to unrealistic appearances and potential performance issues during model training.

To prevent this, a table removal step is applied to all cropped volumes in the 4D CTA dataset. This step uses a simple threshold-based method that identifies high Hounsfield Unit values near the inferior end of the scan. A morphological filter is then applied to ensure that only flat, table-like regions are removed without affecting nearby anatomical structures.

This function, named `remove_table`, is already available as part of the preprocessing utilities in the DiffDeepDRR code base and is integrated into the simulation pipeline.

The ASOCA dataset does not include visible table artifacts, so this step is not applied to those volumes.

Coronary Artery Segmentation

The accurate segmentation of coronary arteries is a critical prerequisite for guidewire simulation. The objective here is to isolate the coronary artery tree with sufficient geometric fidelity to allow centerline extraction and subsequent trajectory sampling. Since guidewire paths are constrained to the vascular lumen, the quality of the coronary segmentation directly determines the anatomical realism and validity of the simulation.

For the 4D CTA dataset, segmentation is initially performed using the `coronary arteries` task of TotalSegmentator, which is optimized for detailed labeling of coronary branches. This task produces a multi-class mask that includes major arteries such as the left anterior descending, left circumflex, and right coronary artery. Although TotalSegmentator provides a strong baseline, the raw segmentation output of some patients can be insufficient for precise simulation purposes due to several limitations:

- Incomplete or fragmented branches,
- Over-segmentation of non-coronary regions (e.g., nearby arteries or noise),
- Presence of small disconnected components and high-frequency artifacts.

To solve these problems, a manual post-processing stage is applied using *3D Slicer* [36], an open-source medical imaging platform. The TotalSegmentator output served as an initial mask, which is refined through the following procedures:

- Removal of irrelevant branches and noise: Small, disconnected structures or mislabeled arteries not part of the coronary arteries are manually excluded.
- Bridging of disconnected segments: Disconnected vessel parts are manually connected.

These refinements significantly improved the anatomical quality and continuity of the coronary masks, making them suitable for centerline extraction and route generation.

Finally, the post-processed coronary artery masks are saved separately for each side of the heart. Two masks are created and stored in different files for left and right coronary arteries. The reason for doing this is to know whether the resulting routes belong to the left or right coronary artery and to be able to use appropriate angles in the DRR generation process, thus ensuring that the guidewire is always visible.

All these operations are not applied to the ASOCA dataset because this dataset already includes segmentation of the left and right coronary arteries.

Centerline Extraction

In medical image analysis, a centerline is defined as a one-dimensional curve that passes through the medial axis of a tubular anatomical structure such as a blood vessel or airway [37]. It provides a compact yet informative representation of the object's geometry, preserving both its path and topological connectivity. Centerlines are widely used for applications like vessel quantification, path planning, and anatomical modeling [38]. In this study, centerline extraction is performed by applying following steps:

- Skeletonization and Graph Construction: Each coronary mask is first converted into a smooth binary volume to suppress small artifacts. A 3D skeleton is then computed to reduce the vessel to its topological core. This skeleton is interpreted as a graph, where nodes represent bifurcation or endpoint locations, and edges represent vessel segments.
- Centerline Extraction and Surface Reconstruction: A surface mesh of the vessel is reconstructed from the binary mask. Anatomical inlets and outlets are detected by intersecting surface normals with skeleton endpoints. These points serve as the origin and destination for centerline generation. The resulting polylines are geometrically smoothed, resampled at regular intervals (e.g., every 0.1 mm), and refined to ensure a consistent representation of vessel radius and shape. The final output is saved in `.vtp` format for further use.
- Automation and Quality Control: The entire process is applied in an automated fashion to all patients. Optional visualization modules are used to verify mesh quality, centerline accuracy, and branching structure.

All operations are performed using the *Vascular Modeling Toolkit*, a widely used open-source library for vessel analysis in medical imaging [39]. The resulting centerlines

preserved both anatomical structure and geometric fidelity, enabling accurate and clinically relevant path sampling in the following stages of the simulation pipeline. The centerline extraction process is based on an existing in-house script, rather than being implemented from scratch.

Route Extraction

After obtaining coronary centerlines, the next step is to define guidewire routes, possible paths that a real guidewire might follow during an interventional procedure. These routes form the basis of simulating realistic navigation scenarios in the coronary arteries. The process involved several steps:

- Identification of Candidate Points: Terminal points on the centerline graph are identified as candidate start and end points. This ensured that the extracted routes represented full vessel traversals rather than short, internal segments.
- Graph Construction and Route Search: To extract possible routes from the centerline, a graph is constructed where each sampled point on the centerline acts as a node. If two terminal points are directly connected in the original centerline, an edge is created between them. The edge is weighted using the Euclidean distance between the points. Once this graph is built, all possible non-repeating (simple) paths between each start and end point pair are computed. This is done using a standard depth-first search strategy provided by the *NetworkX* library [40].
- Length-Based Filtering: Each candidate path is evaluated based on its total physical length. Paths shorter than 5 cm are discarded, as they are unlikely to provide meaningful motion for guidewire simulation. Only sufficiently long and anatomically meaningful routes are kept.
- Saving and Classification: Each accepted path is saved in a separate `.vtp` file for use in next stages of simulation pipeline. Additionally, routes are categorized based on their origin: routes from the left coronary artery tree are saved under a `left/` directory, and those from the right tree under `right/`.

Vessel Mask Generation for DRR Rendering

In clinical X-ray fluoroscopy, blood vessels are typically visible only during brief intervals when a contrast agent is actively injected [41]. Even then, only a subset of the vascular network becomes visible at any given time, depending on the injection site, flow rate, and imaging phase. In contrast, CTA volumes contain high-resolution static representations of the entire vascular system. As a result, using the raw volume directly

in DRR rendering leads to unrealistic projections where too many vessels appear simultaneously, contradicting the sparse appearance observed in real interventions.

To address this inconsistency and simulate clinically realistic DRRs, binary masks of vessels are needed to control the visibility of vessels during DRR generation. However, rather than enhancing vessel visibility, these masks are used primarily to suppress selected vascular structures, and making them not visible.

For the 4D CTA dataset, composite vessel masks are constructed by combining multiple anatomical segmentations produced by TotalSegmentator. The following components are included:

- Systemic arteries: Obtained from the `total` segmentation task, capturing major vessels such as the aorta and iliac arteries.
- Lung vasculature: Obtained using the `lung_vessels` task of TotalSegmentator.
- Cardiac chambers: Extracted using the `heartchambers_highres` task.
- Coronary arteries: Binary masks obtained during the previous process are used.

Then, these obtained binary masks are combined together.

For the ASOCA dataset, which provides a static CTA volume per patient, the same anatomical components are segmented directly using TotalSegmentator, and the pre-provided coronary masks are integrated into the final vessel map using the same label hierarchy.

These vessel masks can be used as a visibility control mechanism during DRR synthesis: by excluding or up-weighting certain vessel regions from the attenuation model, the rendered images can be adjusted to simulate realistic clinical conditions. Specifically, instead of revealing the entire vascular network, which would appear unnatural, the masks are used to hide undesired structures, resulting in appearances more representative of real fluoroscopic sequences.

Dataset-Specific Implementation Notes

While the overall preprocessing pipeline is consistent across datasets, certain steps are selectively applied based on data availability. For the 4D CTA dataset, segmentation, cropping, orientation normalization, and route extraction are performed, as raw volumes required anatomical isolation and temporal processing. In contrast, the ASOCA dataset already includes cropped volumes and high-quality coronary masks, eliminating the need for additional segmentation or cropping. Route extraction is applied to both datasets; however, in 4D CTA, it is limited to the first time phase ($t = 0$) due to the temporal nature of the data, with deformations applied to simulate motion in later phases.

3.2 Registration and Deformation Field Extraction

Image registration is a fundamental concept in medical image analysis. It refers to the process of aligning two or more images so that relevant anatomical structures match spatially. In clinical imaging, registration is often used to compare scans taken at different times, from different patients, or using different imaging modalities [42]. In this study, registration is primarily used to observe how anatomy deforms over time by aligning volumetric CTA scans taken at different time points in the cardiac cycle.

Registration techniques can be divided into two main categories: rigid (only allowing translation and rotation) and non-rigid (also allowing local deformations). For dynamic organs like the heart, where tissue undergoes complex motion throughout the cardiac cycle, non-rigid (or deformable) registration is necessary. Previous work on non-rigid registration using B-spline deformation supports its effectiveness for dynamic anatomical structures [43].

Given a fixed image I_F (e.g., a CTA volume at end-diastole) and a moving image I_M (e.g., a volume at end-systole), the goal of registration is to find a spatial transformation $\mathcal{T}(\mathbf{x})$ that maps each voxel $\mathbf{x}$ in I_M to its corresponding location in I_F.

The output of this process is a *deformation field*, a dense vector field $\mathbf{d}(\mathbf{x})$ indicating how each voxel in the moving image must shift to align with the fixed image [43]:

$$\mathcal{T}(\mathbf{x}) = \mathbf{x} + \mathbf{d}(\mathbf{x}) \tag{3.1}$$

This deformation field captures the anatomical motion between two time-points and forms the foundation for simulating temporally dynamic image sequences [44]. By warping static anatomical data using these fields, one can propagate coronary arteries, guidewires, soft tissues around the heart region and CTA volumes through time, generating realistic cardiac motion.

To perform this registration and extract deformation fields, intensity-based methods are employed using the open-source tool *Elastix* [45, 46]. Elastix supports both rigid and non-rigid models (e.g., B-spline), and optimizes a similarity metric (e.g., mutual information) over a multi-resolution pyramid. Once registration parameters are estimated using Elastix, the resulting transformation model is saved in parameter files. These parameters are then passed to *Transformix*, which is a part of Elastix, can either warp new image data according to the computed transformations, or export a dense displacement field that describes voxel-wise motion between the registered volumes.

This methodology ensures temporally smooth and anatomically faithful deformations across time phases of the 4D CTA dataset. It also enables spatial alignment between the 4D CTA and ASOCA datasets, which is critical for consistent analysis and training in downstream simulation pipelines.

3.2.1 Rigid Registration

Rigid registration is one of the most fundamental techniques in medical image alignment. It assumes that the objects being registered undergo only rigid-body transformations. This transformation is composed of translations and rotations but does not allow for scaling or shearing.

Mathematically, a rigid-body transformation in 3D space is represented as a special case of an affine transformation. For a point $\mathbf{x} = (x_1, x_2, x_3)$ in the original image space, the transformed point $\mathbf{y} = (y_1, y_2, y_3)$ is given by [47]:

$$\begin{aligned} y_1 &= n_{11}x_1 + n_{12}x_2 + n_{13}x_3 + n_{14} \\ y_2 &= n_{21}x_1 + n_{22}x_2 + n_{23}x_3 + n_{24} \\ y_3 &= n_{31}x_1 + n_{32}x_2 + n_{33}x_3 + n_{34} \end{aligned} \tag{3.2}$$

This affine transformation is often represented in matrix form as:

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ 1 \end{bmatrix} = \begin{bmatrix} m_{11} & n_{12} & n_{13} & n_{14} \\ m_{21} & n_{22} & n_{23} & n_{24} \\ m_{31} & n_{32} & n_{33} & n_{34} \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ 1 \end{bmatrix} \tag{3.3}$$

Equations (3.2) and (3.3) are adapted from Ashburner and Friston [47].

In this study, rigid registration is applied to align CTA volumes acquired from different datasets into a common anatomical space. This alignment enables subsequent steps, such as deformation transfer, to operate in a unified spatial domain.

3.2.2 B-Spline Registration

Non-rigid registration is essential when aligning anatomical structures that undergo local deformations, such as the heart or vasculature. One of the most effective models for capturing such deformations is the B-Spline Free-Form Deformation model [48]. This technique overlays a mesh of control points onto the image domain and defines the deformation field as a weighted sum of basis functions centered around these control points.

Let the image volume domain be:

$$\Omega = \{(x, y, z) \mid 0 \leq x \leq X, 0 \leq y \leq Y, 0 \leq z \leq Z\} \tag{3.4}$$

and let Ψ denote a uniform control point lattice with resolution $n_x \times n_y \times n_z$ and spacing δ. For a point (x, y, z) in the image, its deformation is given by:

$$T(x,y,z) = \sum_{l=0}^{3}\sum_{m=0}^{3}\sum_{n=0}^{3} B_l(u)B_m(v)B_n(w)\Psi_{i+l,j+m,k+n} \tag{3.5}$$

where:

$$\begin{aligned} i &= \left\lfloor \frac{x}{n_x} \right\rfloor - 1, \quad j = \left\lfloor \frac{y}{n_y} \right\rfloor - 1, \quad k = \left\lfloor \frac{z}{n_z} \right\rfloor - 1, \\ u &= \frac{x}{n_x} - \left\lfloor \frac{x}{n_x} \right\rfloor, \quad v = \frac{y}{n_y} - \left\lfloor \frac{y}{n_y} \right\rfloor, \quad w = \frac{z}{n_z} - \left\lfloor \frac{z}{n_z} \right\rfloor. \end{aligned} \tag{3.6}$$

The functions $B_l(\cdot)$ represent the cubic B-Spline basis functions, defined as:

$$\begin{aligned} B_0(u) &= \frac{(1-u)^3}{6}, \\ B_1(u) &= \frac{(3u^3 - 6u^2 + 4)}{6}, \\ B_2(u) &= \frac{(-3u^3 + 3u^2 + 3u + 1)}{6}, \\ B_3(u) &= \frac{u^3}{6} \end{aligned} \tag{3.7}$$

Equations (3.4)–(3.7) are taken from Wu, Lan, J. Wang, et al. [48].

Since B-Spline interpolation is controlled locally, changing a single control point affects only a small surrounding region. This makes the model computationally efficient and scalable to large datasets. The locality of the deformation also helps maintain anatomical realism in the resulting transformations.

In this work, B-Spline registration aligns each time-step of the 4D CTA sequence to the reference time phase. The control grid spacing (δ) and optimization parameters are set empirically (e.g., 16 voxels spacing, max 500 iterations). This generates dense, smooth deformation fields that capture cardiac motion. The resulting deformation fields serve to warp volumes, masks, and simulated guidewire volumes consistently across time.

3.2.3 Usage of Registration in This Study

To simulate realistic cardiac motion on both 4D CTA and static CTA volumes from the ASOCA dataset, a two-step registration pipeline is implemented using SimpleITK and Elastix.

The process begins with a rigid registration between a static CTA volume from the ASOCA dataset (used as the fixed image) and the first time step of a 4D CTA volume (used as the moving image). This step brings both datasets into a same spatial space, allowing temporal motion extracted from the 4D CTA series to be applied meaningfully

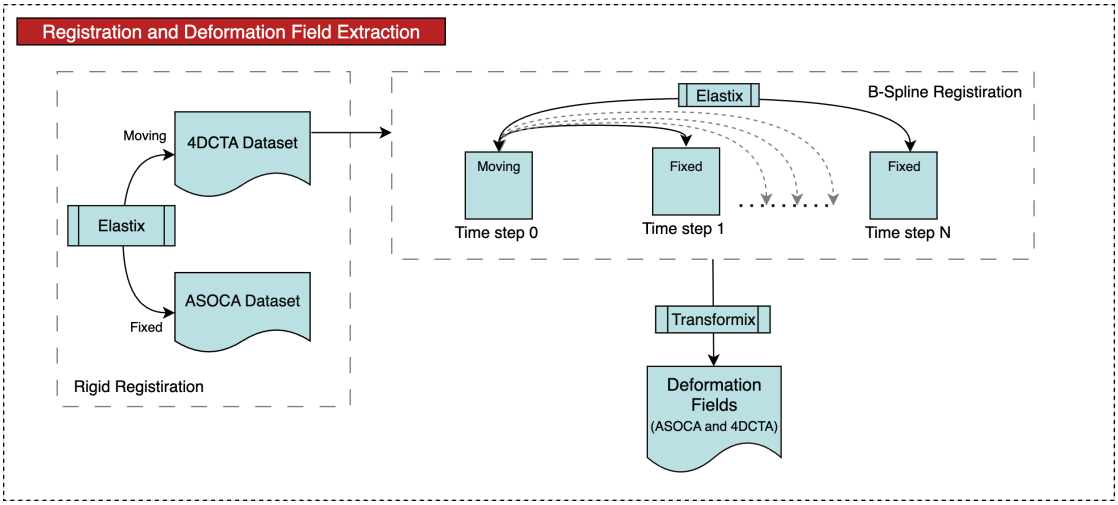


Figure 3.2: Overview of registration and deformation field extraction process.

to the ASOCA dataset. The rigid transformation parameters are computed using Elastix with the default rigid registration parameter map provided by Elastix, and the resulting transformation is saved for aligning two datasets in the same space.

Once alignment is established, the next step is computing temporal deformations within the 4D CTA series. For each patient, the first time step is registered to all subsequent time steps using B-spline registration, which captures non-rigid deformations such as those caused by cardiac motion. In this stage, each fixed image corresponds to a time step $t > 0$, while the first time step remains the common moving image. The resulting B-spline transform for each time step is passed to Transformix, which is configured to compute the dense deformation field. These deformation fields, represented as 3D vector images, saved in `NIfTI` format. At the end of this process, the deformation fields that can transform the first timestep into all subsequent time-steps are calculated for each patient. Twenty deformation fields are obtained for each patient.

For each ASOCA patient, a 4D CTA patient is selected using modulo indexing to ensure balanced use. The deformation fields generated from the 4D CTA sequence are then spatially compatible with the ASOCA volumes, thanks to the initial rigid alignment. As a result, realistic temporal motion extracted from 4D data can be applied to otherwise static CTA scans in ASOCA.

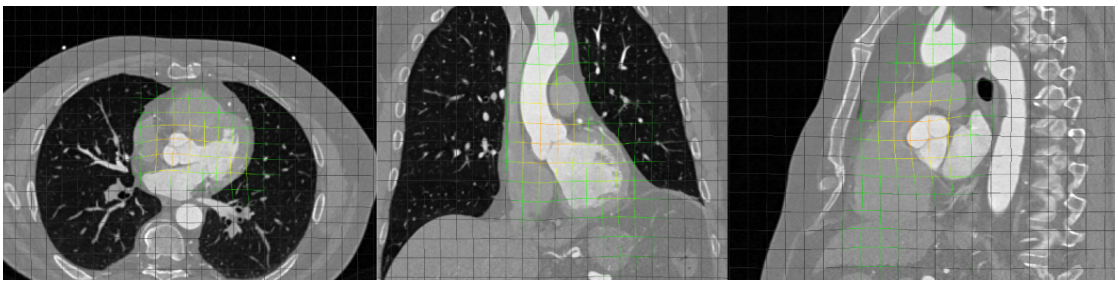


Figure 3.3: An example deformation field visualized on axial, coronal and sagittal view, for 4D CTA dataset.

3.3 DRR Dataset Generation

This section describes the dataset generation pipeline used to simulate synthetic X-ray images (DRRs) with guidewire movement and heart motion. The pipeline integrates coronary artery routes, deformation fields, and guidewire models to generate DRR sequences for use in detection and tracking models.

3.3.1 Simulation of Guidewire

The guidewire motion is simulated using an in-house guidewire simulation tool developed within the DHM research group. This tool loads a predefined 3D coronary artery route (in `.vtp` format) and simulates the motion of a guidewire tip along this path. The guidewire is characterized by a defined tip length and radius. At each time step, the tip region is represented as a sequence of points sampled from the route, which are then transformed into voxel space using the spatial transformation matrix of the CTA volume.

The position of the guidewire tip along the route is determined by a parameter called *moving step*, which specifies how far along the path the guidewire has advanced at a given time. Based on this position, a tip segment with the defined length and radius is selected to generate the mask.

A binary mask is created using a KDTree algorithm [49], marking all voxels within the given radius from the tip centerline. If deformation fields are provided, they are applied to the guidewire tip points during mask generation to ensure spatial consistency with the deformed anatomical structures.

This binary mask is then inserted into the CTA volume as Tungsten material, which is commonly used in real guidewire tips due to its high radiopacity [50]. By using the same material in the simulation, the generated DRR sequences produce a realistic appearance of the guidewire, closely mimicking how it would appear in real fluoroscopic imaging.

3.3.2 Applying Motion of Heart

To simulate cardiac motion throughout the DRR sequence, patient-specific deformation fields are applied to a static CTA volume. Each field represents the displacement from the reference phase ($t = 0$) to a later stage of the cardiac cycle. Applying these fields sequentially enables the generation of DRR frames that reflect different phases of heart motion.

Since the available deformation fields represent deformations from the reference time step ($t = 0$) to each subsequent phase, the CTA volume at $t = 0$ is preserved as a reference. For each time step, this reference volume is re-used, and the corresponding deformation field is applied to generate a temporally deformed version of the volume. This approach ensures consistency and avoids error accumulation across steps.

To support this, the `update_volume()` function is implemented inside the Projector module of DiffDeepDRR. This function warps the CTA volume, material grid, and mask grid using the given 3D deformation field. A coordinate grid normalized to the range $[-1, 1]$ is first generated based on the reference volume's resolution. The deformation field is then scaled accordingly to ensure compatibility with PyTorch's grid sampling [18].

The warped volume is then obtained using `grid_sample`, which performs trilinear interpolation for continuous fields such as the CTA volume and material grid. This ensures that anatomical structures are smoothly deformed. To further reduce interpolation artifacts and produce visually consistent DRRs, a light Gaussian smoothing is also applied.

Additionally, the guidewire tip mask is updated at each time step by applying the same deformation field, maintaining temporal consistency between the guidewire and the surrounding anatomy. This is essential for realistically modeling the interaction of the guidewire with the moving heart during DRR generation.

3.3.3 Generating DRR Sequences

In this study, realistic guidewire sequences are generated using the DiffDeepDRR framework, where X-ray simulation is performed from CTA volumes. For each patient, a 3D CTA volume and a predefined guidewire route are provided as inputs. Guidewire movement and heart motion is simulated through a loop, in which the guidewire is advanced step by step along the coronary path using the *moving step* parameter, while cardiac motion is involved by applying patient-specific deformation fields which are extracted in the previous steps of the pipeline.

The deformation fields, which are extracted from the patient's 4D CTA data, represent one full cardiac cycle. At each iteration of the loop, the next deformation field is applied

to the reference CTA volume using the `update_volume()` function of Projector module of DiffDeepDRR. The number of available deformation fields determines the temporal resolution of the sequence, while the video duration is selected to reflect a realistic number of heartbeats within that time span. The generated sequence is designed to simulate a reasonable number of cardiac cycles according to its duration. To achieve multiple heartbeats within a single video, the deformation fields are applied in an iterative loop, allowing the cardiac motion pattern to repeat over time. In this way, temporally realistic DRR sequences are produced.

At each time step, the guidewire tip mask is generated using the *Guidewire Simulation* tool. This binary mask is integrated into the volume using the Projector module's `update_material()` function, in which a high-attenuation region of Tungsten is inserted at the mask location. A guidewire radius of 0.35 mm and a length of 3.5 cm are used, which reflect common specifications of guidewires used in coronary interventions [51]. Tungsten is selected as the material due to its high attenuation coefficient, which ensures visibility in DRR images.

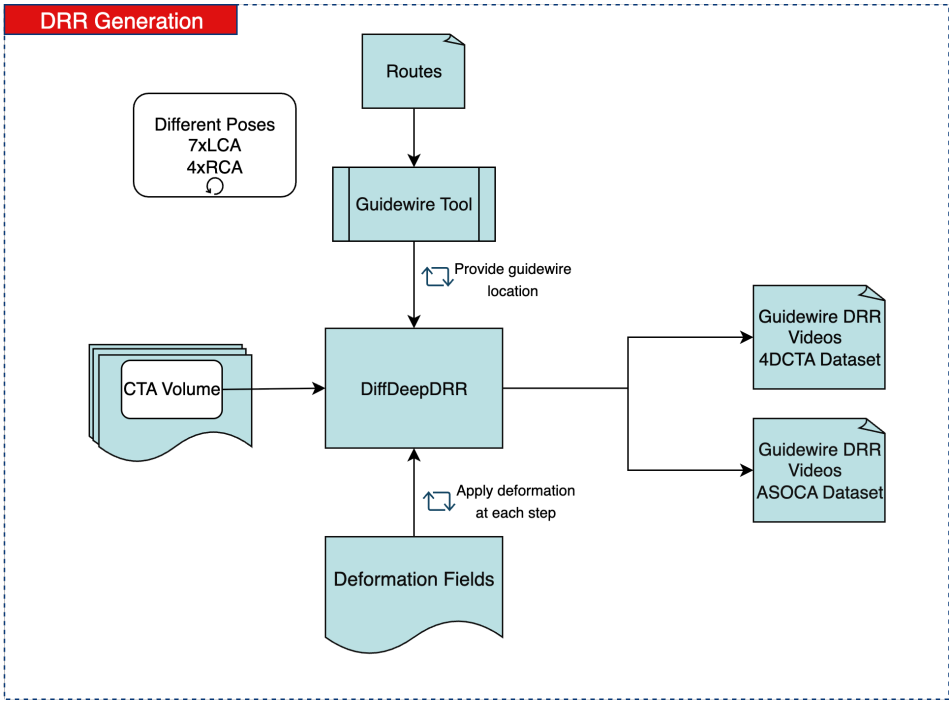


Figure 3.4: Overview of DRR generation pipeline

The vascular structures are also incorporated into the simulation through a pre-computed vessel mask. This binary mask is used to suppress vascular visibility in

the resulting DRRs, which increases the visual contrast between the guidewire and surrounding anatomy. As a result, the realism of the simulated images is improved to better resemble real fluoroscopic data, where vascular structures are not visible unless contrast agents are used.

In this study, the `CA-Angio` device type is selected, which is one of several available device types within the DiffDeepDRR framework. This particular setup is considered suitable for coronary interventions, as it reflects clinically realistic imaging geometry and X-ray spectrum properties. The X-ray device pose is defined using six parameters:

- α, β, γ: rotations around the X, Y, and Z axes (yaw, pitch, roll),
- t_x, t_y, t_z: translations along the X, Y, and Z axes.

To ensure optimal visibility of the guidewire and to increase dataset diversity, multiple camera angles are defined for each coronary route. Specifically, 7 different poses are used for *left* coronary routes, and 4 different poses are used for *right* coronary routes. In each case, DRR sequences are generated from various angles, allowing the guidewire to be captured under multiple clinically relevant projections.

The volume is resampled with a spacing of 0.4 mm. Although GPU memory usage is increased as a result, higher spatial resolution is achieved, and more detailed anatomical structures are observed in the generated DRRs. In addition, the visibility of the guidewire is significantly improved. Since the guidewire is a very thin structure, it is prone to appearing fragmented when coarser spacing values are used. By using fine spacing, a more continuous and realistic guidewire appearance is ensured.

This full procedure is repeated across both datasets (ASOCA and 4D CTA), for each patient (26 in total), all coronary routes, and every defined pose. As a result, 178 sequences and a total of 26017 images are obtained from the 4D CTA dataset. 773 sequences and a total of 147956 images are obtained from the ASOCA dataset. In total 951 DRR sequences with simulated heat motion and guidewire movement are obtained.

Importantly, each DRR image is produced together with its corresponding guidewire mask, enabling the creation of a fully **annotated simulation dataset**. This allows the generated data to be directly used in supervised learning pipelines for guidewire detection and tracking tasks.

Figure 3.5 shows example frames from two generated DRR sequences. The **top row** presents three frames sampled from a guidewire sequence in the ASOCA dataset, where the guidewire is navigating through in a *left coronary artery*. The **bottom row** displays frames from a 4D CTA dataset sequence showing the guidewire progressing through a *right coronary artery*. Different pose parameters are used in each case to simulate clinically meaningful X-ray viewpoints, enhancing the visibility and variability of guidewire appearances in the dataset.

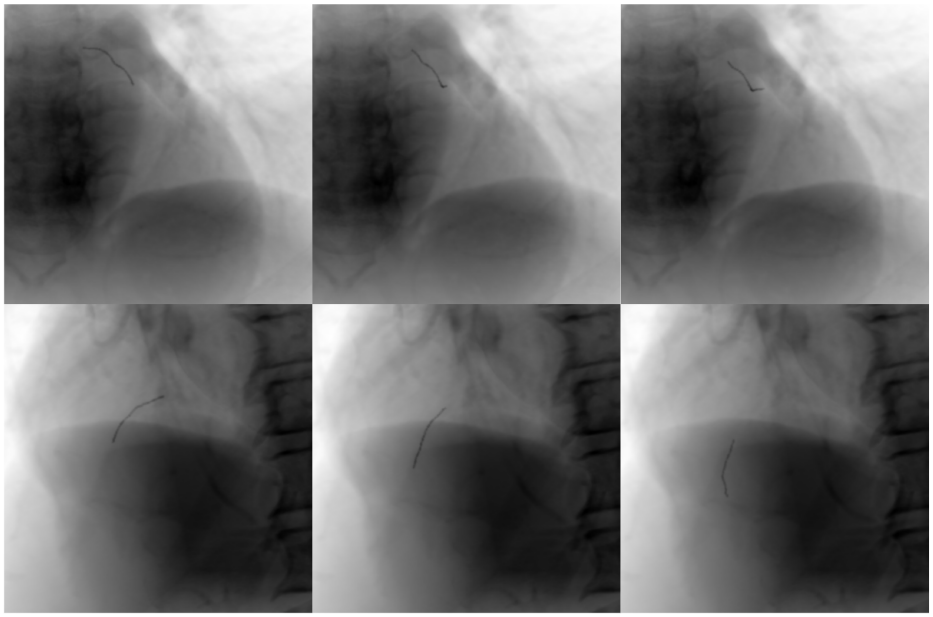


Figure 3.5: Example DRR frames with simulated guidewire from ASOCA and 4D CTA datasets.

3.3.4 Obtaining DRR Dataset and Resampling

After the DRR generation process, all guidewire sequences are manually reviewed to ensure visual quality. Although the X-ray device pose parameters are carefully selected based on the coronary route type (left or right), there are still cases where the guidewire is not consistently visible throughout the sequence or is distorted due to camera angles. The most reliable method to detect and eliminate such problematic sequences is through manual inspection. If these sequences are not removed, cases where the guidewire is invisible or appears in anatomically implausible shapes may introduce false positives during model training. As a result, a total of 258 sequences are discarded during this quality control phase.

After filtering, 693 sequences and over 110,000 individual DRR frames are retained. For a single-class detection and tracking task, this amount of data is considered unnecessarily large and computationally inefficient. In addition, redundant or overly similar frames are likely to exist, which may reduce the effectiveness of training. Therefore, it is important that the dataset is resampled in a way that preserves diversity while reducing scale [52, 53].

To achieve this, an uniform resampling strategy is applied. From each remaining se-

quence, 15 frames are selected at equal intervals, ensuring that the temporal progression and anatomical variation within the sequence are maintained [53]. This approach helps preserve diversity across patients, routes, and poses, and prevents certain sequences from dominating the dataset.

Importantly, this reduction not only minimizes dataset size but also introduces temporal gaps between frames, which is beneficial when training models such as QDTrack. In QDTrack, appearance-based learning is performed using a limited number of reference-adjacent frames, typically within a small temporal window (e.g. k=5). Therefore, having noticeable appearance or positional variations of the guidewire across nearby frames can lead to more effective learning, as the model is encouraged to capture diverse representations within a short temporal span.

As a result of this resampling process, 10,365 images are obtained from the filtered sequences. These images are divided into training, validation, and test sets using a 60%-20%-20% sequence-wise split. Each frame is accompanied by a guidewire mask and a bounding box annotation, forming a fully labeled and temporally structured dataset suitable for training detection and tracking models. Notably, the ASOCA and 4D CTA datasets are not treated as separate sources during model development. Rather, they are merged into a single unified dataset.

3.4 Obtaining Guidewire Tracking Models

In state-of-the-art approaches for object tracking, *tracking-by-detection* paradigm is the mainstream approach [22]. In this setup, object detection is first learned as a separate task, and during inference, detected objects are associated across frames using tracking algorithms [54]. This allows temporal continuity to be inferred based on frame-wise detections, often without requiring explicit temporal supervision. In this study, the same paradigm is adopted: guidewire detection is treated as a prerequisite, and tracking is achieved by associating per-frame detections using dedicated tracking models.

3.4.1 YOLOv8

YOLOv8 is part of the *You Only Look Once* family of object detectors, which are popular for being both fast and reliable. Developed by Ultralytics, it follows a single-stage detection approach, meaning it finds and classifies objects at once [20]. This makes it especially useful for real-time applications like guidewire detection in fluoroscopic sequences. YOLOv8 is a good fit for guidewire detection task, provides features such as:

- Anchor-free architecture: The model does not use fixed anchor boxes. Instead, it

learns to predict object positions and sizes directly, which makes training more lightweight and more flexible.

- Decoupled head: It separates classification and box regression into different parts of the network, helping it learn each task more effectively.

In this study, YOLOv8 is employed as guidewire detection model. It's inference speed is high and is can be integrated with tracking algorithms, making it a practical choice for medical use where timing matters [55].

However, YOLOv8 alone does not track objects over time, it only sees one frame at a time. To build full trajectories of the guidewire, it needs to be combined with a tracking algorithm that can link detections across frames. This tracking step is explained in the following sections.

Advantages

- Fast inference suitable for real-time medical applications: YOLOv8 delivers fast predictions, making it ideal for time-sensitive domains like medical applications.
- Lightweight: Thanks to its lightweight architecture, the model can be trained with limited computational resources and relatively small datasets.
- Flexible integration: YOLOv8 can be integrated with tracking algorithms and domain adaptation pipelines.

Limitations

- Detection quality in challenging cases: As a one-stage detector, YOLOv8 may struggle to accurately detect hard-to-identify or low-contrast guidewire regions, especially in cluttered or noisy images.
- No use of temporal information during training: Since the model processes each frame independently, it cannot utilize temporal information during the training. If the associated tracking module is not carefully tuned, this can lead to frequent ID switches.

Preparing the Data

The YOLOv8 expects a specific folder structure and annotation format. The dataset is organized into two main folders: `train` and `val`, each containing two subdirectories named `images` and `labels`. All training images are stored as individual png files inside the `images` folders, while corresponding object annotations are saved as text files in the

`labels` folders. Each annotation file has the same name as the image it refers to (but with a `.txt` extension), and contains one or more lines, where each line represents a bounding box in the format: `<class_id> <x_center> <y_center> <width> <height>`, with all coordinates normalized to the range $[0, 1]$ with respect to image width and height.

The raw annotations are originally available as binary mask images, where the guidewire mask appeared as dark regions (pixel value near 0) and the background as white (pixel value near 255). To convert these into the required bounding box format, all contours in each mask are extracted using *OpenCV* [56], and the smallest bounding rectangle enclosing the guidewire is computed. This bounding box then normalized and written into the corresponding YOLO label file.

The dataset is split into train, val, and test subsets using a 60%-20%-20% ratio. Importantly, this split is performed on the basis of sequences rather than individual frames to preserve temporal continuity within each set. Each sequence, representing a continuous series of frames from a single guidewire path, is treated as a single unit and assigned entirely to one of the three splits. This setup ensures that no two frames from the same guidewire sequence are placed into different splits. By keeping entire sequences within a single subset (train, val, or test), the dataset avoids any overlap across splits, maintaining a clear separation between training and evaluation data.

Training YOLOv8

The YOLOv8 family consists of multiple model variants—`yolov8n`, `s`, `m`, `l`, and `x`—that offer trade-offs between accuracy and computational cost. These models share the same architecture but differ in network width and depth, allowing users to select an appropriate version based on task complexity and hardware constraints. All variants come with pre-trained weights on the Common Objects in Context (COCO) dataset [57], which contains 80 object categories, and are widely used as strong baselines for fine-tuning in domain-specific tasks [58].

In this study, `yolov8n`, the smallest model in the family, is selected. This choice is motivated by the nature of the problem: a single-class detection task involving only one or two instances per frame. In such cases, using larger models does not necessarily lead to better performance but increases training and inference times. The compact size of `yolov8n` makes it best fit for fast experimentation and potential real-time deployment, while still offering accurate guidewire detection when trained on high-quality data.

The model training relied on the `YOLO.train()` function provided by Ultralytics. The optimizer is set to `"auto"`, which means the framework selects the optimizer and learning rate based on the dataset size and total number of iterations. With these data, it chooses the AdamW optimizer [59] with an initial learning rate of 0.002 and a

momentum of 0.9.

Training ran for 50 epochs using a batch size of 16 and an image size of 512×512 pixels. Early stopping is enabled with a patience value of 5 epochs. To ensure reproducibility, the random seed is fixed at zero.

The built-in augmentation settings includes augmentation techniques such as horizontal flipping, color jittering, and mosaic augmentation. All outputs such as trained weights, logs, and evaluation metrics are saved to a dedicated project folder for analysis and future use.

BoT-SORT for Tracking with YOLOv8

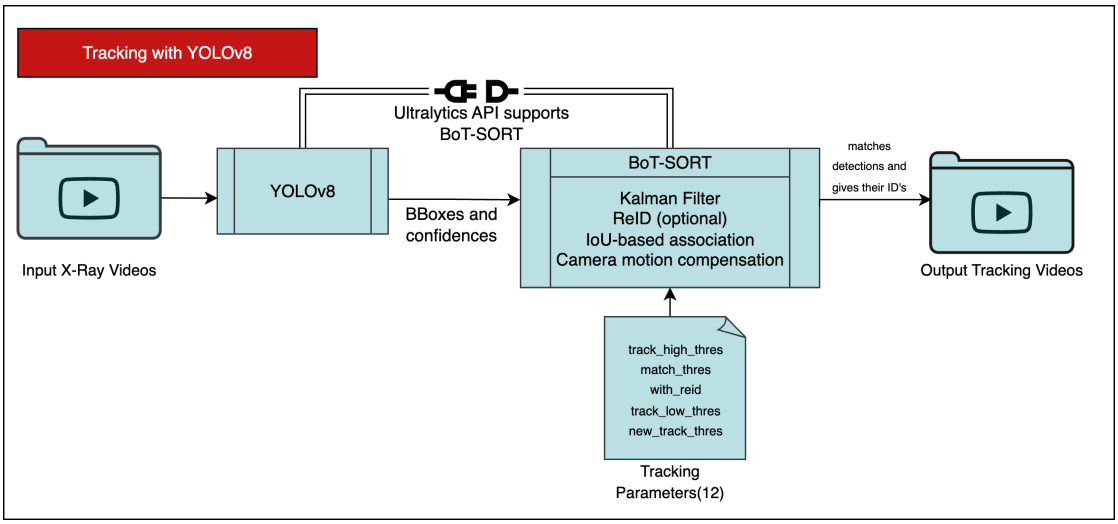


Figure 3.6: Overview of the YOLOv8 + BoT-SORT tracking model.

While YOLOv8 provides frame-level object detection, it does not inherently support temporal tracking. To assign consistent identities to guidewire detections across frames, a tracking algorithm is required. In this study, the BoT-SORT tracker is used to extend YOLOv8's output into full tracking trajectories.

BoT-SORT is an advanced tracking-by-detection algorithm that builds upon the original SORT framework. Some of it's key features are:

- **Kalman Filtering** for motion prediction,
- **Intersection over Union (IoU)-based association** to match detections with existing tracks,
- **Re-identification (ReID)** module (optional) for appearance-based matching,

- **Camera motion compensation**, although this is not relevant in static-view fluoroscopy.

In practice, bounding boxes and confidence scores predicted by YOLOv8 are passed to the BoT-SORT module. The tracker links these detections over time based on spatial overlap and optional appearance embeddings. As a result, each detected guidewire tip receives a consistent ID across video frames.

The Ultralytics API natively supports integration with BoT-SORT, allowing for seamless end-to-end detection and tracking pipelines [58]. A set of hyper-parameters such as `track_high_thres`, `match_thres`, and `with_reid` can be configured to control the tracker's behavior.

The combined system enables reliable multi-frame tracking of guidewires, turning per-frame detections into temporally coherent trajectories suitable for downstream analysis.

In this work, tracking is performed without the ReID module. This decision is based on the nature of guidewire appearances in fluoroscopic images—guidewires lack consistent color, texture, or other high-level visual features that typical ReID models rely on. Their shapes are often highly variable and affected by projection angle and deformation, which may cause ReID-based matching to introduce more errors than benefits.

Camera motion compensation is also not enabled, as the X-ray datasets used in this study consist of sequences acquired from fixed angles, making such correction unnecessary. Additionally, enabling this feature causes significant decrease in inference performance without considerable tracking improvements in static-camera settings.

All other tracking hyper-parameters are carefully tuned for scenarios involving one or two guidewires per sequence. Parameter values are selected empirically to balance ID consistency and track stability, ensuring robust tracking performance suitable to the characteristics of the medical imaging data used in this study.

Inference and Optimizing Tracking Parameters

The default tracking parameters provided by BoT-SORT are typically designed for pedestrian or vehicle tracking applications. However, guidewire tip tracking in X-ray fluoroscopy differs significantly from such scenarios. Therefore, all tracker parameters are carefully re-optimized to meet the specific needs of this domain (see Table 3.1). In most cases, only one or two guidewire tips appear in the scene, and due to their flexible structure, they deform according to the vascular path they follow. This leads to significant changes in shape across frames. Accordingly, the parameters are adjusted to handle noisy movements, avoid false positives, and maintain identity consistency.

Parameter	Description	Selected Value	Rationale for Selection
`track_high_thresh`	Threshold for the first association during tracking. Determines how confidently a detection is matched to an existing track.	0.6	Increased to ensure that only high-confidence guidewire detections are matched to existing tracks, reducing identity switches.
`track_low_thresh`	Threshold for the second, fallback association when the high-confidence match fails. Uses more relaxed criteria.	0.4	Kept moderately low to allow recovery from occasional missed detections, while still avoiding overly uncertain associations.
`new_track_thresh`	Threshold to start a new track when a detection cannot be matched to any existing one.	0.5	Ensures that a new guidewire track is created only when the detection is reasonably confident, reducing false positives.
`track_buffer`	Number of frames a lost track is retained before being removed.	60	Allowing temporary occlusions to be tolerated without breaking the track.
`match_thresh`	Threshold for matching detections to existing tracks. Higher values make the matching stricter.	1.0	Set to maximum to ensure stricter associations and minimize ID switches.
`fuse_score`	Enables fusion of detection confidence with IoU distance during association.	True	Helps stabilize tracking by considering both spatial proximity and model confidence.
`gmc_method`	Global motion compensation method, used to account for camera motion.	None	Disabled since camera motion is negligible or fixed in X-ray fluoroscopy sequences.
`with_reid`	Enables appearance-based matching to improve tracking through occlusions.	False	Disabled because the shape of the guidewire can vary greatly between frames, it can harm ReID accuracy.

Table 3.1: BoT-SORT tracker hyper-parameters and justifications tailored for guidewire tracking [58].

Instead of relying on appearance-based ReID matching—which can be unreliable due to the constantly changing shape of the guidewire—spatial-based matching is used. BoT-SORT predicts the likely position of an object in the next frame using previous trajectories. A detection is matched with an existing track if it overlaps with the predicted location by a sufficient IoU and exceeds the `track_high_thresh` value. This setting helps reduce identity switches caused by misleading appearance cues. Each parameter in Table 3.1 is tuned with these domain-specific characteristics in mind, enabling more stable and accurate guidewire tracking.

Tracking inference parameters are tuned using the validation splits of the single guidewire real X-ray dataset and the simulation DRR dataset. The same tracking parameters are used when testing all YOLO + Bot-SORT models on different data.

3.4.2 QDTrack

QDTrack is a state-of-the-art multi-object tracking framework based on the tracking-by-detection paradigm. Unlike traditional approaches that rely heavily on motion models, QDTrack performs association primarily in the appearance (embedding) space, which makes it particularly suitable for applications where motion cues are unreliable or objects move irregularly.

The training pipeline of QDTrack consists of two branches: a key frame and a reference frame, both processed through a shared feature extractor and an embedding head. The network generates proposal boxes for both frames, from which embeddings are extracted. These embeddings are used in a multi-positive contrastive learning setup, where multiple matching pairs (positive samples) of the same object are pulled closer in the embedding space, and negative samples (different objects) are pushed further apart. This results in an embedding space that better discriminates between different object identities.

At inference time, QDTrack processes each input frame to obtain object detections and their corresponding embeddings. The similarity between these embeddings and those from recent frames is computed using a bi-directional softmax similarity metric. Other similarity functions such as cosine or softmax can also be used. Matching is performed through nearest neighbor search in the embedding space, and the tracker assigns identities based on embedding similarity, without relying on explicit motion modeling. This makes QDTrack particularly suitable for scenarios where motion patterns are unreliable.

A key component of QDTrack is the use of a memory mechanism, where embeddings of objects from the last k frames are stored. This enables the model to perform temporal matching over a short history, which improves identity assignment by smoothing over occasional detection noise or occlusion.

Advantages.

- Appearance-based tracking allows robust identity assignment even when objects do not follow easily predictable trajectories.
- Can be used with different detectors.
- Uses temporal information not only during inference, but also in training.

Limitations.

- Appearance-based tracking can be problematic for flexible and shape-changing objects like guidewire.

- Without explicit motion modeling, the tracker may struggle to disambiguate objects with similar appearance but diverging trajectories.
- Inference parameters should be tuned carefully, otherwise there is a possibility of high numbers of false positives and id switches.

MMTracking

MMTracking is an open-source framework developed by the OpenMMLab team [60], designed to support a wide range of video tracking tasks. It includes tools for training and evaluating different types of tracking models, such as multi-object trackers, single-object trackers, and video object detectors.

In this work, MMTracking is used to train the QDTrack model. One of the main reasons for choosing MMTracking is that it includes built-in support for QDTrack, which makes it easier to get started without writing a custom training pipeline from scratch. At the same time, it still provides flexibility to adjust settings and experiment with different configurations.

Another important advantage of MMTracking is that it brings many widely used tracking models into one unified framework. If methods like OC-SORT [61] or Tracktor [62] need to be tried later, they can be trained and evaluated within the same framework. This keeps everything more organized and removes the need to work with multiple separate repositories or environments. MMTracking also supports different dataset formats like MOTChallenge [63] and COCO, which makes it more flexible with a high variety of datasets.

Overall, it offers a practical and scalable solution for training tracking models in a consistent way.

Preparing the Data

The QDTrack model is trained using the standard Multiple Object Tracking (MOT) format supported by MMTracking. Each sequence (i.e., guidewire video) is stored in a separate subdirectory under either the `train` or `val` folder. This structure makes it straightforward to link each frame to its corresponding video and allows MMTracking to process temporally coherent sequences.

The data is accompanied by COCO-style annotation files, `train.json` and `val.json`. These files include structured entries for `images`, `annotations`, `videos`, and `categories`. Each image is annotated with its `frame_id`, `video_id`, and bounding box coordinates, allowing QDTrack to associate detections across frames and keep track of object identities across different frames of a sequence.

To comply with the MOT format requirements, each video sequence directory also includes two essential files: `gt.txt` and `seqinfo.ini`. The `gt.txt` file stores the ground truth bounding boxes in a frame-wise manner, following the MOTChallenge format specification. The `seqinfo.ini` file provides metadata about the sequence, including frame rate, resolution, image format, and sequence length. These files allow the dataset to be used seamlessly with MMTracking and other MOT-compatible evaluation tools.

The train-validation split mirrors the one used for the YOLOv8 experiments. Specifically, sequences assigned to the training set in YOLO remain in the training set here as well, and the same applies to the validation split. This consistent partitioning ensures that no temporal or patient-level data leakage occurs between training and evaluation phases. Thus, QDTrack and YOLO models are trained with the same data and fairness is ensured when comparing model performances.

Training QDTrack

The QDTrack model is trained using a configuration adapted from the official MOT17 challenge configuration for QDTrack provided by the MMTracking library. The overall architecture combines a Faster R-CNN detector [64] with a ResNet-50 backbone [65] and Feature Pyramid Network neck [66]. The backbone is initialized with pretrained weights from the COCO dataset, which improves convergence and generalization.

Several data augmentation techniques are applied during default training pipeline to improve model robustness. These include standard data augmentation methods like resizing, cropping, flipping, and normalization. These augmentations simulate variations in viewpoint, illumination, and object scale.

The training pipeline uses stochastic gradient descent as the optimizer, with a learning rate of 0.001, momentum of 0.9, weight decay set to 0.0001 and epoch is set to 4. These values are commonly used defaults in many object detection tasks and are adopted directly from the original MOT17 QDTrack configuration provided by MMTracking.

During training, the detector learns to localize guidewire instances in each frame, while the tracking component focuses on learning robust appearance embeddings that are consistent across time. The actual tracking behavior parameters such as memory length and matching thresholds are configured separately at inference time.

Overall, the training setup balances practical training efficiency with the goal of learning temporally consistent identity features, and builds upon a well-tested pipeline tailored for multiple object tracking tasks.

Parameter	Description	Selected Value	Rationale for Selection
`init_score_thr`	Threshold to initialize a new track. Only detections with confidence above this value can start a new track.	0.99	Enforced high confidence to avoid initializing tracks on false positives.
`obj_score_thr`	Threshold to update existing tracks. Detections below this value are discarded for updating.	0.9	Ensures track consistency by updating only with very confident detections, helping reduce identity switches.
`match_score_thr`	Similarity threshold for associating a detection with an existing track.	0.1	Low value allows matching even under appearance changes, which is common for deformable guidewires.
`nms_conf_thr`	Confidence threshold for filtering overlapping boxes during NMS.	0.5	A balanced value to retain high-confidence detections while suppressing redundant boxes.
`with_cats`	Whether to match objects based on class labels.	False	Disabled because the dataset contains only a single object category.
`match_metric`	Embedding similarity metric used for association. Options include `bisoftmax`, `softmax`, and `cosine`.	`cosine`	Cosine similarity yielded more stable performance in associating guidewire segments under deformation.

Table 3.2: QDTrack tracker hyper-parameters and rationale tailored for guidewire tracking.

Inference and Optimizing Tracking Parameters

The selection of QDTrack tracker hyper-parameters (Table 3.2) is made with careful consideration of the model's detection behavior. QDTrack's detection head demonstrates strong performance in identifying true positives, rarely missing relevant objects. However, it is also prone to producing high-confidence false positives, especially in noisy or ambiguous areas of DRR images. In practice, most true positives have confidence scores close to 1.0, yet many false positives also exceed 0.9. To prevent unreliable detections from affecting the tracking pipeline, the `init_score_thr` and `obj_score_thr` thresholds are set to 0.99 and 0.9, respectively, ensuring that only the most confident detections are used to initialize or update tracks.

Due to the elastic and deformable nature of guidewires, their appearance can vary significantly across frames, making identity association difficult. To address this, the `match_score_thr` is set to a low value of 0.1, allowing the tracker to maintain identity continuity even when the visual appearance changes. For the similarity metric used during association, `cosine` distance is selected after evaluating alternatives such as `softmax` and `bisoftmax`. While `bisoftmax` is the default one for QDTrack, it performed poorly in this context. Since the training data contains only a single object per frame, the

model is able to learn positive pair associations but lacked sufficient negative samples to learn meaningful separations. In contrast, `cosine` similarity offered more robust and stable performance under deformation, and is adopted because of this reason.

Tracking inference parameters are tuned using the validation splits of the single guidewire real X-ray dataset and the simulation DRR dataset. The same inference parameters are used when testing all QDTrack models on different data.

3.5 Unsupervised Domain Adaptive Detection Pipeline

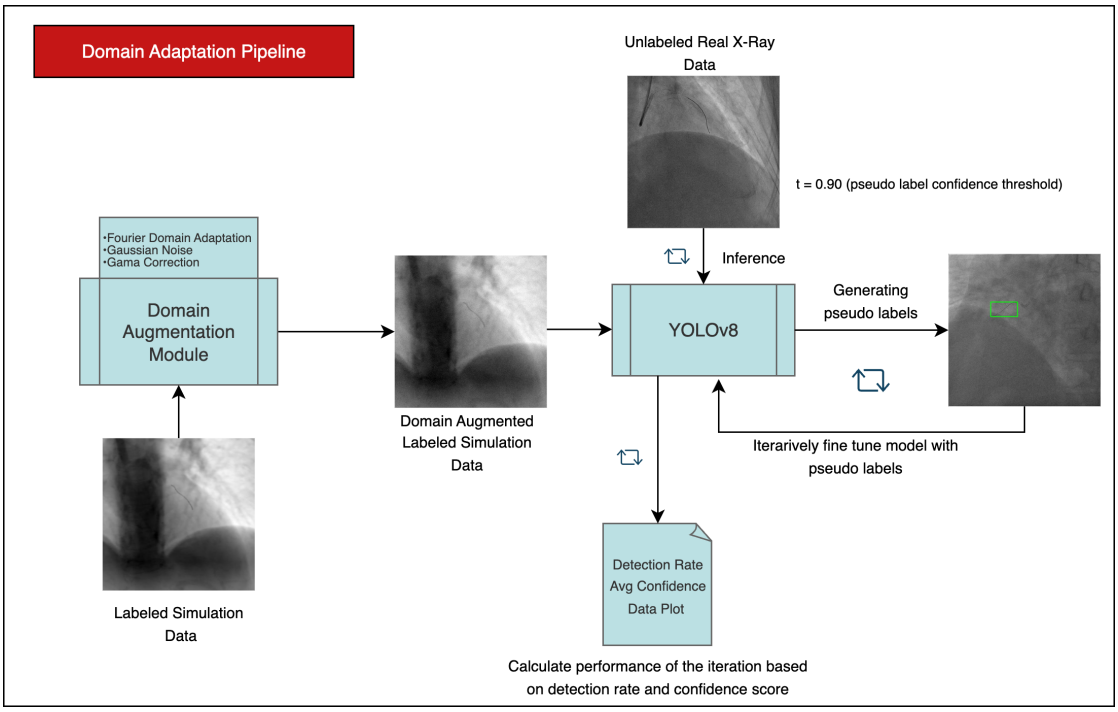


Figure 3.7: Overview of proposed domain adaptation pipeline with pseudo labeling.

Unsupervised Domain Adaptation (UDA) addresses the problem of transferring models trained on a labeled source domain to an unlabeled target domain. This is particularly important when the two domains exhibit significant differences in appearance, a scenario commonly referred to as the domain gap [67]. Closing this gap is essential for models trained on synthetic data to generalize effectively to real-world clinical images.

The proposed pipeline adopts an unsupervised, domain-adaptive detection strategy to bridge this gap between simulation-based DRR images and real X-ray fluoroscopy

data. Specifically, it focuses on adapting an object detection model, YOLOv8 trained on labeled simulation data to perform reliably on unlabeled real X-ray images. This strategy does not rely on any real annotations and leverages pseudo labeling to progressively adapt the model.

An important motivation for this approach lies in the structure of the overall tracking pipeline. Since the tracking models used in this study, are fundamentally built upon frame-wise detections, any improvement in the robustness of the detector directly translates into better tracking performance. By minimizing the domain gap at the detection stage, the consistency and accuracy of guidewire trajectories in real X-ray videos are significantly enhanced.

The pipeline follows an iterative procedure: first, a domain augmentation module transforms simulation images to more closely resemble real X-rays. A YOLOv8 detector is trained on these augmented images using ground-truth labels. Once trained, the model is applied to real X-rays, and high-confidence detections are collected as pseudo labels. These pseudo-labeled samples are then used to fine-tune the same pretrained detector in the next iterations. It is important to note that during fine-tuning, only the pseudo-labeled real data is used; the original simulation data is not merged back into the training set. During each cycle, both the detection performance and average confidence scores are monitored to determine the most promising iteration.

Throughout the domain adaptation process, the configuration used for training and fine-tuning the YOLOv8 model remains identical to that described in the *Training YOLO* subsection. This ensures a controlled evaluation of the domain adaptation effect without introducing additional variables from hyperparameter tuning. It also allows for a fair comparison between models, as all models are trained under identical configurations.

The high-level overview of this iterative procedure is illustrated in Figure 3.7. The following subsections provide a detailed explanation of each pipeline component, including domain augmentation techniques, pseudo labeling strategy, and best iteration selection criteria. In the remainder of this study, this unsupervised domain adaptation pipeline is referred as the *UDA pipeline*.

3.5.1 Domain Augmentation Methods

Although DRR images provide a flexible way to simulate X-ray projections from CTA volumes, their appearance often differs from real X-ray fluoroscopy due to differences in noise patterns, contrast levels, and intensity distributions. These differences can create a domain gap that negatively affects model generalization [68].

To reduce the visual gap between synthetic and real X-ray images, a domain augmentation module is implemented. This module takes the base DRR images and applies a series of transformations to make them look more like real X-ray images. The goal

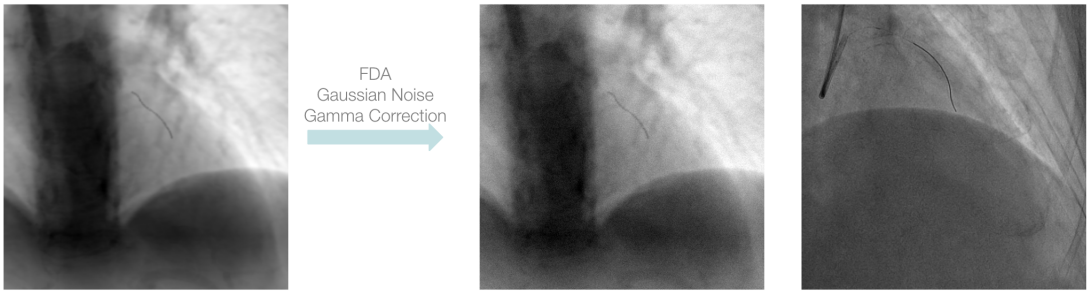


Figure 3.8: Examples from domain augmentation pipeline. From left to right: base DRR image, domain-augmented DRR, and real X-ray.

here is not to generate perfectly realistic images, but to make the appearance of the synthetic data close enough to help the model learn features that generalize better across domains. At the same time, these augmentations increase the visual diversity of the training data, enabling detection models to perform more reliably under a wider range of imaging conditions. The augmentation pipeline includes the following steps:

- **Fourier Domain Adaptation (FDA) [69]:** Low frequency components from real X-rays are partially transferred into the DRRs to bring the overall texture and contrast closer to clinical images.
- **Gaussian Noise:** Adds variations at the pixel level, similar to sensor noise in real world scenarios.
- **Gamma Correction:** Adjusts brightness and contrast, helping DRRs better match the lighting variations seen in real data.

The greatest advantages of these methods are that they are very lightweight and do not require any training.

FDA also requires images from the target domain. Therefore, one image from each sequence is selected from real-world X-ray datasets to represent the target domain, and the low-frequency features of these sample X-ray images are transferred to DRR images during the FDA process.

The effect of these augmentation processes is visualized in Figure 3.8, where a base DRR image (left) is compared to its augmented version (middle) and a sample real X-ray (right). As seen in the figure, domain augmentation reduces the visual gap, especially in terms of texture and noise distribution.

By applying these transformations consistently across the training set, the resulting model is encouraged to learn domain-invariant features, making it more robust when

tested on real clinical data. However, to avoid completely altering the data, these domain augmentation strategies are applied to a 30% of the data rather than the entire data. This preserves most of the original DRRs and increases the diversity of the dataset by including examples that reflect the characteristics of the target domain using domain augmentation strategies.

3.5.2 Choosing Real X-ray Dataset for Pseudo Labeling

For the purpose of pseudo labeling, a subset of real X-ray data is selected from the training split of the single-guidewire dataset. Specifically, 60% of this split, corresponding to 3,770 images, is used as the unlabeled target domain on which pseudo labels are generated. This selection provides a sufficiently diverse yet manageable dataset for iterative fine-tuning.

Importantly, the double-guidewire real X-ray dataset is excluded from all stages of pseudo labeling and model training. It is reserved only for final testing to evaluate the generalization capabilities of the trained models across diverse datasets. This separation ensures that the domain adaptation strategies are tested not only within the distribution of the training set but also in a different, more complex clinical context, thereby offering a stronger assessment of cross-dataset robustness.

3.5.3 Pseudo Labeling Strategy

Pseudo labeling is a widely used strategy in semi-supervised learning, and it has become particularly effective in the context of UDA for object detection tasks [27, 70, 71]. The core idea behind pseudo labeling is to leverage a model trained on labeled source domain data to produce artificial labels for previously unlabeled samples from the target domain. These pseudo labels are then treated as ground truth in subsequent rounds of training, enabling the model to gradually adapt to visual characteristics of the new domain without requiring any real annotations.

In this study, pseudo labeling strategy is used to adapt a guidewire detection model trained on synthetic DRR images to perform reliably on real X-ray images, which differ in appearance due to domain-specific noise, contrast, and anatomical variability.

To begin, a YOLOv8 detector trained exclusively on simulated DRR data is used to generate bounding box predictions on real X-ray frames. In each iteration, only the predictions with confidence scores above a strict threshold of 0.9 are accepted as valid pseudo labels. This high-confidence filtering step serves for these purposes: it helps reduce the inclusion of false positives that can mislead the model and ensures that retraining is performed with reliable, high-quality detections. As a result, the model is exposed to trusted samples from the target domain and begins to learn its visual

distribution.

The pseudo labeling process is carried out iteratively over the course of 10 iterations. In the early stages, few detections meet the confidence threshold due to the domain gap between the synthetic and real data. However, as the model progressively adapts to the target domain, the quantity and of the pseudo-labeled samples improve. This iterative refinement process forms a self-improving learning cycle: as the model adapts to the target domain, it generates increasing number of reliable pseudo labels, which in turn help improve its performance in the next iterations.

In each iteration, the pseudo-labeled data obtained from the previous round is carried over to form the training dataset for the next iteration. In addition to these previously labeled samples, the model is used to infer and assign new labels to the remaining unlabeled images. These new detections are then added to the growing pseudo-labeled dataset, allowing the model to iteratively expand its coverage over the target domain while avoiding redundant labeling. This strategy helps prevent randomness by ensuring that each iteration only generates pseudo-labels for previously unseen samples. By avoiding repeated or arbitrary selection, the model focuses on progressively harder examples, and its labeling decisions are cumulatively built upon reliable outcomes from earlier iterations.

Given that the model is already proficient at detecting guidewires in simulated images, only minor adjustments are needed to adapt it to the new domain, eliminating the need for extensive retraining. The pseudo-labeled data primarily serve to fine-tune an already capable detector, enabling it to adjust to domain-specific visual cues with relatively few updates. To prevent overfitting and reduce unnecessary computation, the `patience` parameter in YOLOv8's training configuration is set to 5. This means that training will automatically stop if the model's validation performance does not improve for five consecutive epochs. In all iterations of pseudo labeling, early stopping is triggered, indicating that the model is able to quickly converge and suggesting that its learned features already generalize well. This efficient convergence reinforces the assumption that the base model provides a strong initialization, and that pseudo labeling mainly acts as a lightweight domain adaptation mechanism.

3.5.4 Choosing the Best Iteration

Figure 3.9 illustrates the evolution of pseudo-labeling across ten iterations, following two key metrics: the number of pseudo-labeled images (blue line), and their average confidence scores (orange line).Fixed pseudo label confidence threshold of 0.9 is shown with green dashed line and fixed number of total X-ray images is shown with gray dashed line. The process begins with a base model and continues iteratively, with each new model fine-tuned on the previously generated pseudo-labeled data and used to

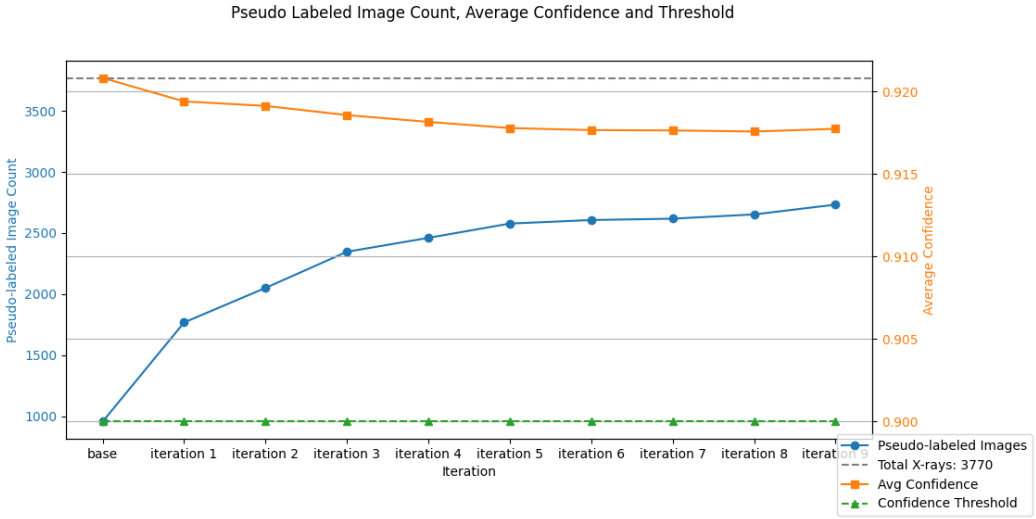


Figure 3.9: Pseudo labeled data count and average confidence plot over iterations.

label more samples from the target domain.

As seen in the plot, the number of pseudo-labeled images steadily increases at each iteration, indicating a gradual and consistent adaptation of the model to the target domain. This progressive growth reflects that the model becomes increasingly capable of assigning confident labels to previously unlabeled examples. The average confidence shows a slight decrease over iterations. This trend is expected, as the iterations progress, more challenging examples remain and they may cause slight drop in the average confidence of pseudo labels.

In the last iteration, the model produces the highest number of pseudo-labeled images, while maintaining a strong average confidence. This suggests that not only has the model managed to label a significant portion of the target dataset, but it has also done so with acceptable reliability. The combination of high label count and solid average confidence indicates that the model has converged to its maximum pseudo-labeling capacity on the available data. Therefore, the model obtained at the last iteration is selected for evaluation on real X-ray datasets, as it is trained on the most comprehensive, and still reliable pseudo-labeled dataset.

Although average confidence is treated as a secondary decision factor, it plays a critical role in ensuring label quality. In scenarios where more labels are generated at the cost of confidence, earlier iterations may be preferred to avoid noisy supervision. However, in this case, both metrics favor the last iteration, making it the optimal point for selecting the final model for further testing.

4 Experiments and Results

This chapter presents a comprehensive evaluation of the proposed guidewire tracking pipeline across multiple datasets. The goal is to assess the effectiveness of domain adaptation strategies and compare model performance between simulated (DRR) and real X-ray images. All models are tested under consistent evaluation settings to ensure fair comparisons.

The performance of the models is evaluated across three distinct datasets, each chosen to highlight a different aspect of model behavior:

- **Test split of the DRR dataset:** Used to confirm that the models perform well in their source domain, ensuring that the training process is valid and that the models effectively learn to track guidewires under simulated conditions.
- **Test split of the single guidewire real X-ray dataset:** Serves to evaluate how well the models generalize to real X-ray images. This comparison highlights the domain gap between synthetic and real data and provides insight into the effectiveness of the proposed domain adaptation strategy.
- **Double guidewire dataset:** Designed to test model robustness under more complex scenarios involving multiple objects. Additionally, it assesses cross-dataset generalization by measuring how well the models, trained or adapted on one dataset, perform on a different real-world dataset.

4.1 Evaluation Metrics and Tools

For tracking evaluation, the *TrackEval* [72] framework is used—one of the most widely adopted open-source toolkits for multi-object tracking evaluation. TrackEval supports various standard metrics used in the literature. Among them, the following are selected for detailed analysis in this study:

- **Higher Order Tracking Accuracy (HOTA) [73]:** A balanced metric that jointly evaluates detection and association accuracy. For a given matching threshold α, it is defined as:

$$\text{HOTA}_\alpha = \sqrt{\text{DetA}_\alpha \cdot \text{AssA}_\alpha} \tag{4.1}$$

where DetA_α refers to detection accuracy and AssA_α to association accuracy at threshold α. DetA quantifies how well the tracker detects objects, while AssA reflects its ability to maintain consistent identities over time.

The overall HOTA score is obtained by integrating over a range of thresholds:

$$\text{HOTA} = \int_0^1 \text{HOTA}_\alpha \, d\alpha \approx \frac{1}{19} \sum_{\alpha \in \{0.05, 0.1, \ldots, 0.95\}} \text{HOTA}_\alpha \tag{4.2}$$

- **Multi-Object Tracking Accuracy (MOTA) [74]:** Evaluates the tracking performance by jointly penalizing false negatives, false positives, and identity switches over all frames. It is formally defined as:

$$\text{MOTA} = 1 - \frac{\sum_t (FN_t + FP_t + IDSW_t)}{\sum_t GT_t} \tag{4.3}$$

where FN_t, FP_t, and $IDSW_t$ are the false negatives, false positives, and identity switches at time t, respectively; GT_t is the number of ground truth objects at time t.

- **IDF1 [75]:** Measures the ratio of correctly identified detections over the average number of ground-truth and computed detections. It is the F1-score of the ID precision and recall:

$$\text{IDF1} = \frac{2 \cdot \text{IDTP}}{2 \cdot \text{IDTP} + \text{IDFP} + \text{IDFN}} \tag{4.4}$$

IDTP denotes the number of correctly identified detections with the correct identity assignment. IDFP refers to detections that are matched to a ground-truth object but assigned an incorrect identity. IDFN indicates ground-truth detections that are missed by the tracker or assigned an incorrect identity. These metrics collectively assess identity consistency in tracking performance.

- **Frames Per Second (FPS):** Indicates the inference speed of the tracking pipeline, reflecting its real-time applicability.

4.2 Evaluated Models

To evaluate the impact of different data and models, several models are trained using a combination of synthetic and real datasets. These models provide reference points to compare the effectiveness of domain adaptation and pseudo labeling:

- **YOLOv8 trained on base DRR data:** Serves as the baseline model trained only on simulated DRR images.
- **YOLOv8 trained on domain-augmented DRR data:** Trained on DRR images processed through domain adaptation techniques such as FDA, gamma correction and Gaussian noise.
- **YOLOv8 trained on annotated real X-ray data:** Acts as an oracle, trained using fully annotated real X-ray images. It is used to benchmark how closely domain adaptation methods can approach performance with real labeled data.
- **QDTrack trained on base DRR data:** A tracking-by-detection model trained purely on simulated DRR sequences without domain adaptation. It serves as a baseline reference for evaluating tracking performance in the absence of real data or adaptation.
- **QDTrack trained on domain-augmented DRR data:** Used to evaluate how well temporal models perform when trained on synthetic but domain-augmented data. Serves as a reference for evaluating tracking models in a simulation-based setting.
- **QDTrack trained on annotated real X-ray data:** Another reference tracking model trained using manually annotated real X-ray sequences to assess upper-bound tracking performance on real data.
- **YOLOv8 model obtained from the proposed unsupervised domain adaptation pipeline:** This model is first trained on domain-augmented DRR data and then iteratively fine-tuned using high-confidence pseudo labels generated from unlabeled real X-ray images. It reflects one of the core contributions of this study.

The pseudo labeling process exclusively uses the training split of the single-guidewire real X-ray dataset. For the models trained on labeled real X-ray data, both the training and validation splits of the same dataset are utilized. The double-guidewire real X-ray dataset is excluded entirely from both training and pseudo labeling phases and is reserved solely for testing. This setup enables an unbiased evaluation of cross-dataset generalization performance.

Given that the proposed unsupervised domain adaptation pipeline relies on a YOLO-based detector, an ablation study is performed on YOLO based tracker to assess the separate effects of domain augmentation and pseudo-labeling processes. Further details and results of this study are provided in the following sections.

4.3 Results on Simulation Dataset

Models trained on the DRR dataset are first evaluated on the test split from the same domain. This step aims to confirm that the models have been properly trained and calibrated for their own domain, and that they can perform well when no domain gap is present. High performance in this setting indicates that the models have effectively captured the characteristics of the simulated data and are able to operate under ideal, in-domain conditions.

Method	**Training Data**	**HOTA**	**MOTA**	**IDF1**	**FPS**
YOLO + BoT-SORT	DRR	79.128	97.664	98.828	~80
QDTrack	DRR	90.634	97.713	98.845	~10

Table 4.1: Tracking performance of models on simulation dataset.

Table 4.1 presents the tracking results of models trained on base DRR data. Both models exhibit great performance across all tracking metrics, confirming their effectiveness when tested within the same simulation domain. While QDTrack achieves higher HOTA due to its dedicated temporal training and two-stage detector, the YOLO-based tracker offers a major advantage in inference speed, reaching 80 FPS compared to QDTrack's 10 FPS. This substantial difference makes the YOLO-based approach significantly more practical for real-time medical applications. For this reason, the domain adaptation pipeline proposed in this study is built upon the YOLO.

4.4 Ablation Study on Domain Adaptation Pipeline

As mentioned, the proposed domain adaptation pipeline is composed of several stages. First, domain augmentation techniques such as FDA and Gaussian noise injection are applied to the synthetic DRR images to reduce the visual gap between simulated and real X-ray data. Following this, an iterative pseudo-labeling process is carried out to gradually adapt the model to the target domain using high-confidence predictions. To better understand the contribution of each component in this pipeline, an ablation study is conducted and presented in this section. This ablation study is conducted using

both the single-guidewire and double-guidewire real X-ray datasets, allowing for a comprehensive evaluation of each component's performance across different scenarios.

Method	DRR	Domain Augmentation	Pseudo-Labeling	HOTA	MOTA	IDF1
YOLO + BoT-SORT	✓	✗	✗	56.340	42.991	69.285
YOLO + BoT-SORT	✓	✓	✗	62.065	63.178	74.466
YOLO + BoT-SORT	✓	✗	✓	73.013	86.168	92.717
YOLO + BoT-SORT	✓	✓	✓	**76.672**	**92.523**	**96.176**

Table 4.2: Ablation study on the effect of domain augmentation and pseudo-labeling, tested on the single guidewire real X-ray dataset.

As can be seen in Table 4.2, each step of the proposed pipeline plays an important role in improving tracking performance. Applying domain augmentation or pseudo-labeling individually already brings noticeable gains. However, when both are combined, the improvement becomes clearly significant. These findings highlight the strength of the full pipeline, showing that combining domain augmentation with iterative learning leads to more stable and reliable tracking performance.

Method	DRR	Domain Augmentation	Pseudo-Labeling	HOTA	MOTA	IDF1
YOLO + BoT-SORT	✓	✗	✗	25.332	-11.224	19.259
YOLO + BoT-SORT	✓	✓	✗	62.380	69.388	83.333
YOLO + BoT-SORT	✓	✗	✓	62.940	71.088	83.101
YOLO + BoT-SORT	✓	✓	✓	**70.789**	**87.415**	**93.285**

Table 4.3: Ablation study on the effect of domain augmentation and pseudo-labeling, tested on the double guidewire real X-ray dataset.

Table 4.3 shows that a similar trend holds for the more challenging double guidewire case. Both techniques contribute positively on their own, and when used together, they lead to further improvements. These results are in line with the single guidewire findings, reinforcing the value of combining domain augmentation and pseudo-labeling in the proposed pipeline.

4.5 Results on Single Guidewire Real X-Ray Dataset

To evaluate the real-world effectiveness of the trained models and assess their ability to generalize beyond the simulation domain, experiments are conducted on the single guidewire real X-ray dataset. This dataset represents the target domain in the unsupervised domain adaptation setting and serves as the main benchmark to validate the success of the proposed pipeline. Since the ultimate goal of this study is to enable

accurate guidewire detection and tracking in unlabeled real X-ray data by bridging the domain gap between simulated and real world data, the performance on this dataset provides the most meaningful insight into the practical applicability of the models. All experiments are performed on the test split of the single guidewire real X-ray dataset, which consists of 5 sequences.

Method	**Training Data**	**HOTA**	**MOTA**	**IDF1**	**FPS**
YOLO + BoT-SORT	DRR	56.34	42.991	69.285	**~80**
QDTrack	DRR	58.124	49.907	68.922	~10
YOLO + BoT-SORT	Domain Aug. DRR	62.065	63.178	74.466	**~80**
QDTrack	Domain Aug. DRR	62.203	53.832	74.126	~10
YOLO + BoT-SORT	Annotated Real X-ray	73.047	83.925	91.683	**~80**
QDTrack	Annotated X-ray	**85.143**	**93.084**	94.073	~10
UDA Pipeline	Domain Aug. DRR, Unannotated X-ray	76.672	92.523	**96.176**	**~80**

Table 4.4: Tracking performance of models on the single guidewire real X-ray dataset.

Table 4.4 presents the tracking results on the single guidewire real X-ray dataset. As seen in the table, applying domain adaptation techniques such as FDA and Gaussian noise leads to noticeable performance improvements for both YOLO and QDTrack models compared to their counterparts trained on base DRR data. These improvements are consistent across all tracking metrics (HOTA, MOTA, and IDF1).

Interestingly, the model obtained from the proposed UDA pipeline performs even better than the YOLO model trained on labeled real X-ray data. Although this might seem overly optimistic at first, it is important to consider that the training split of real X-ray dataset used here consists of only 12 sequences. This limited volume may not be sufficient to train a model that generalizes well under diverse conditions. Moreover, real-world data often contains higher variability, which means there may be non-negligible distribution differences between the training and test splits. On the other hand, the UDA pipeline benefits from a simulated dataset, and fine-tuning it with high-confidence pseudo labels allows the model to better adapt to real domain characteristics and generalize better.

Compared to the other oracle model, QDTrack trained on real data, the UDA pipeline performs lower in HOTA but achieves very close performance in MOTA and even outperforms in IDF1. Additionally, since YOLO-based tracking models run at a much higher frame rate (around 80 FPS), they are significantly more suitable for real-time medical applications.

4.6 Results on Double Guidewire Real X-Ray Dataset

The double guidewire real X-ray dataset is used as a challenging benchmark to evaluate the generalization performance of the models across diverse datasets with different characteristics. Unlike the single guidewire dataset, this dataset contains sequences with two simultaneous guidewires, which increases the complexity of the tracking task due to higher number of items to be tracked, possible occlusions, and potential identity switches. Furthermore, this dataset originates from a different clinical source, captured under varying imaging protocols and machine settings. These differences introduce substantial domain shifts in terms of image noise patterns, contrast agent usage, and anatomical variability, making it a difficult test scenario.

Importantly, the double guidewire real X-ray dataset is not included in any part of the training or pseudo labeling process. It is not used during the fine-tuning process with pseudo labels. It is also excluded from the training of models that are trained with labeled real X-ray data used as the oracle. This means that none of the models have seen examples from this source during training. Because of this, the double guidewire dataset is a strong and unbiased benchmark to evaluate how well the models generalize to a different and more challenging dataset. Experiments are performed on the double guidewire real X-ray dataset, which consists of 10 sequences

Method	**Training Data**	**HOTA**	**MOTA**	**IDF1**	**FPS**
YOLO + BoT-SORT	DRR	25.332	-11.224	19.259	**~80**
QDTrack	DRR	64.387	67.687	78.846	~10
YOLO + BoT-SORT	Domain Aug. DRR	62.380	69.388	83.333	**~80**
QDTrack	Domain Aug. DRR	62.760	59.184	73.535	~10
YOLO + BoT-SORT	Annotated X-ray	62.014	71.429	81.905	**~80**
QDTrack	Annotated X-ray	70.421	86.735	89.354	~10
UDA Pipeline	Domain Aug. DRR, Unannotated X-ray	**70.789**	**87.415**	**93.285**	**~80**

Table 4.5: Tracking performance of models on the double guidewire real X-ray dataset.

Table 4.5 presents the tracking results on the double guidewire dataset. YOLO based tracker trained with base DRR shows poor performance on this dataset, which is reasonable considering the challenging nature of double guidewire real X-ray dataset.

The domain-augmented YOLO model shows a considerable improvement in tracking results. Methods like FDA and Gaussian noise help the model handle the noisy and complex backgrounds found in this dataset. These domain-specific variations are better reflected in the training data, making the model more robust.

On the other hand, QDTrack's performance slightly decreases with domain augmentation. One possible explanation is that QDTrack uses a two-stage detector that already

performs well in detecting difficult objects. It tends to favor high recall and avoids missing objects, meaning that domain augmentation did not contribute much on this challenging dataset and might have even added noise that hinders it's accuracy.

The proposed UDA pipeline achieves the best results across all tracking metrics, outperforming even the oracle models trained on labeled real X-ray data. Since those oracle models are trained on a different real X-ray dataset, their generalization to this dataset is limited due to factors like different imaging setups or noise levels. In contrast, the UDA pipeline model, trained on domain-augmented simulation data and fine-tuned using pseudo-labeled real X-rays, generalizes better on unseen different real dataset. With over 70 HOTA, 85+ MOTA, and 90+ IDF1, it demonstrates almost excellent tracking accuracy. Additionally, its inference speed of around 80 FPS makes it highly suitable for real-time medical applications. While models trained on annotated real X-ray data are often limited by factors such as dataset size and diversity, the performance of the UDA pipeline—which combines the advantages of annotated simulation data (e.g., large-scale and diverse image generation) with an effective domain adaptation strategy—can be considered both reasonable and highly successful.

4.7 Implementation Details

All experiments, including DRR generation, training of detection and tracking models, as well as inference steps, are conducted on a system equipped with an NVIDIA Quadro RTX 6000 GPU with 24 GB of memory.

5 Discussion

This chapter provides a broader interpretation of the experimental results and offers critical insights into the effectiveness and limitations of the proposed methods. The discussion is structured into three main parts: evaluation of the simulation pipeline for training data generation process, discussion on model performances with different datasets to see overall effectiveness of proposed methods and potential directions for improving the current work. These sections help bring together the main findings of this study and point out important ideas for future work on guidewire tracking in changing data conditions.

5.1 Discussion on Simulation Pipeline

The simulation pipeline developed in this study helped create realistic DRR sequences that reflect cardiac motion. Deformation fields extracted from 4D CTA volumes are applied to static CTA scans to simulate the heart's motion over time, allowing for more dynamic and realistic image sequences.

The use of DiffDeepDRR made it possible to generate anatomically accurate and flexible DRR projections from various viewing angles. This is important for mimicking realistic fluoroscopy views commonly encountered in clinical environments. As a core component of the simulation pipeline, DiffDeepDRR enables the generation of X-ray–like images using only CTA data, without requiring additional imaging modalities or real fluoroscopy footage. This flexibility allows producing large amounts of realistic and consistent DRR sequences, which are crucial for training and evaluating deep learning models such as guidewire tracking tasks in X-ray fluoroscopy.

Accurate segmentation of the heart and subsequent cropping of the volume reduced computational load while allowing the simulation process to focus on the actual region of interest. Additionally, the segmentation of coronary arteries followed by centerline extraction provided meaningful and anatomically valid paths, which served as useful references to simulate guidewire motion through the vascular structure.

In real clinical settings, guidewires do not always stay centered within the vessel and may come into contact with vessel walls. However, the primary goal of this study is not to replicate every aspect of physical guidewire dynamics, but rather to generate anatomically realistic shapes and appearances that are sufficient for training detection

and tracking models. Since the DRR generation involves a projection from 3D to 2D, some spatial details are inevitably lost during this process. Therefore, simulating highly detailed guidewire motion is not required, as the focus is on generating training data with sufficient anatomical and visual realism rather than replicating exact physical behavior.

5.2 Discussion on Tracking Results

The experimental results in this study provide several insights into the impact of domain adaptation strategies on guidewire detection and tracking performance. First, models trained on domain-augmented DRRs generally outperformed those trained on the base DRR dataset. This demonstrates that even relatively simple augmentation methods such as FDA, Gaussian noise and gamma correction can help bridge the domain gap and improve generalization to real X-ray images.

The proposed pseudo-labeling pipeline, which fine-tunes a model trained on domain-augmented DRRs using high-confidence predictions from unlabeled real X-rays, has proven to be an effective unsupervised domain adaptation strategy. In several cases, the model fine-tuned with pseudo labels performed comparably or even better than models trained directly on annotated real X-ray data. While this might initially seem overly optimistic, it is important to note that real labeled datasets tend to be limited in size. In contrast, simulated data provides larger volume and more controlled variability, which contributes to stronger generalization when combined with high-quality pseudo labels for learning the target domain's features. Additionally, even though both are real-world datasets, there exists a domain gap between the single and double guidewire real X-ray datasets; likely due to differences in imaging equipment, contrast usage, or clinical settings. Combining simulated data with unlabeled real images through domain adaptation not only helps bridge the gap between synthetic and real domains, but also improves generalization across different real datasets.

These results also highlight the value of DRR simulation in medical imaging research. In domains where annotated data is scarce and expensive to obtain, DRRs offer a scalable alternative for training deep learning models. When combined with domain adaptation techniques, simulated data can effectively support downstream tasks such as guidewire detection and tracking, as shown by the strong performance achieved in this study.

5.3 Further Improvements

While the proposed approaches gives solid results, there are still a few directions that can make the simulation more realistic and improve model performance:

- **Making simulations more realistic:** The DRR generation process can be improved by adding more anatomical and clinical details. For example, including full guidewire structures, multiple guidewire tips, catheters, stents, guidewire sheaths, and contrast agent effects would help simulated images better reflect real clinical conditions.
- **Using different CTA volumes and motion patterns:** In this study, motion is added to ASOCA scans by applying deformation fields from a 4D CTA dataset. This approach can be used with other datasets and modalities as well, helping to create more diverse and varied synthetic data.
- **Trying different domain adaptation methods:** More advanced techniques like image translation with diffusion models can help bring simulated and real image styles closer together and reduce the domain gap further.
- **Using transformer-based tracking models:** Transformers might improve tracking performance, especially in difficult scenarios like occlusions, longer sequences, or when multiple guidewires are present.
- **Creating motion with video diffusion models:** Instead of using precomputed motion fields, video diffusion models can be used to generate motion in static DRR images and create realistic DRR sequences.
- **Simulating other regions and devices:** This simulation pipeline can also be applied to other anatomical areas. It may also include different medical tools like catheters, making it possible to train models for multi-class and multi-region tracking tasks.

6 Conclusion

This study presents a complete pipeline for training and evaluating guidewire detection and tracking models using a combination of simulated and real X-ray datasets. The core motivation is to address the domain gap between simulated DRR data generated from CTA and real-world fluoroscopic images, especially in settings where acquiring large-scale annotated real data is difficult or expensive.

To that end, a simulation pipeline is developed to generate realistic digitally reconstructed DRR sequences from CTA volumes. This includes applying deformation fields to static CTA volumes to simulate cardiac motion and extracting vessel centerlines to define possible guidewire trajectories. Using DiffDeepDRR, high-quality DRR projections are generated from various viewing angles, and guidewire volumes are generated based on centerlines of coronary arteries. This setup provided large volumes of anatomically meaningful training data without the need for manual annotation.

To reduce the domain gap, the study explored domain adaptation strategies. First, a series of domain augmentation methods are applied to improve generalization. Then, a pseudo labeling pipeline is proposed, in which a YOLOv8 model trained on domain-augmented DRR is iteratively fine-tuned using high-confidence predictions on unlabeled real X-ray images. This UDA strategy proved effective in bridging the simulation-to-real gap.

The proposed models are evaluated on two different real world datasets: the test split of the single guidewire real X-ray dataset, and the double guidewire real X-ray dataset. Results showed that models trained solely on simulated data struggled on real domains, whereas domain adaptation techniques significantly improved performance. Notably, the pseudo-labeled UDA model achieved performance close to, or even better than, models trained with annotated real X-ray data. Moreover, the YOLOv8-based tracker consistently provided higher inference speeds, making it suitable for real-time clinical applications.

In conclusion, the results show that realistically generated simulation datasets, combined with domain adaptation methods like pseudo labeling, can offer a powerful alternative to relying on large annotated datasets. The pipeline successfully simulates anatomically realistic sequences and leverages unannotated real data to train models that generalize well across domains. This approach has strong potential for accelerating the development of deep learning-based tools in medical imaging tasks where labeled

data is limited.

Future work may explore expanding the simulation framework to include different anatomical regions and other medical devices, as well as incorporating more advanced generative domain adaptation methods. With continued development, simulation-based training strategies can serve as a scalable and efficient path for deploying robust models in real-world clinical settings.

Abbreviations

ASOCA Automatic Segmentation of Coronary Arteries

COCO Common Objects in Context

CT Computed Tomography

CTA Computed Tomography Angiography

DHM Deutsches Herzzentrum

DRR Digitally Reconstructed Radiograph

FDA Fourier Domain Adaptation

FPS Frames Per Second

GAN Generative Adversarial Network

HOTA Higher Order Tracking Accuracy

IoU Intersection over Union

LPS Left Posterior Superior

MOTA Multi-Object Tracking Accuracy

MOT Multiple Object Tracking

NIfTI Neuroimaging Informatics Technology Initiative

ReID Re-identification

UDA Unsupervised Domain Adaptation

List of Figures

3.1 Overview of data preprocessing pipeline. 16
3.2 Overview of registration and deformation field extraction process. . . . 26
3.3 An example deformation field visualized on axial, coronal and sagittal view, for 4D CTA dataset. 27
3.4 Overview of DRR generation pipeline 29
3.5 Example DRR frames with simulated guidewire from ASOCA and 4D CTA datasets. 31
3.6 Overview of the YOLOv8 + BoT-SORT tracking model. 35
3.7 Overview of proposed domain adaptation pipeline with pseudo labeling. 42
3.8 Examples from domain augmentation pipeline. From left to right: base DRR image, domain-augmented DRR, and real X-ray. 44
3.9 Pseudo labeled data count and average confidence plot over iterations. 47

List of Tables

3.1 BoT-SORT tracker hyper-parameters and justifications tailored for guidewire tracking [58]. 37
3.2 QDTrack tracker hyper-parameters and rationale tailored for guidewire tracking. 41

4.1 Tracking performance of models on simulation dataset. 51
4.2 Ablation study on the effect of domain augmentation and pseudo-labeling, tested on the single guidewire real X-ray dataset. 52
4.3 Ablation study on the effect of domain augmentation and pseudo-labeling, tested on the double guidewire real X-ray dataset. 52
4.4 Tracking performance of models on the single guidewire real X-ray dataset. 53
4.5 Tracking performance of models on the double guidewire real X-ray dataset. 54

Bibliography

[1] *Cardiovascular diseases (CVDs)*. Accessed: 2025-07-16. World Health Organization. 2025. URL: https://www.who.int/health-topics/cardiovascular-diseases#tab=tab_1 (visited on 07/16/2025).

[2] H. Robertshaw, L. Karstensen, B. Jackson, A. Granados, and T. C. Booth. "Autonomous navigation of catheters and guidewires in mechanical thrombectomy using inverse reinforcement learning." In: *International Journal of Computer Assisted Radiology and Surgery* 19.8 (2024), pp. 1569–1578.

[3] B. Zhang, M. Bui, C. Wang, F. Bourier, H. Schunkert, and N. Navab. "Real-time guidewire tracking and segmentation in intraoperative x-ray." In: *Medical Imaging 2022: Image-Guided Procedures, Robotic Interventions, and Modeling*. Ed. by C. A. Linte and J. H. Siewerdsen. SPIE, Apr. 2022, p. 33. DOI: 10.1117/12.2611097.

[4] Y.-J. Zhou, X.-L. Xie, G.-B. Bian, Z.-G. Hou, B. Liu, Z.-C. Lai, X.-K. Qu, S.-Q. Liu, and X.-H. Zhou. "Real-Time Guidewire Segmentation and Tracking in Endovascular Aneurysm Repair." In: *Neural Information Processing: 26th International Conference, ICONIP 2019, Sydney, NSW, Australia, December 12–15, 2019, Proceedings, Part I*. Sydney, NSW, Australia: Springer-Verlag, 2019, pp. 491–500. ISBN: 978-3-030-36707-7. DOI: 10.1007/978-3-030-36708-4_40.

[5] D. G. Symes, L. M. McNamara, and C. Conway. "Patient-specific investigation into plaque rupture risk due to catheter tracking during TAVR." In: *Computers in Biology and Medicine* 194 (2025), p. 110558.

[6] G. Litjens, T. Kooi, B. E. Bejnordi, A. A. A. Setio, F. Ciompi, M. Ghafoorian, J. A. van der Laak, B. van Ginneken, and C. I. Sánchez. "A survey on deep learning in medical image analysis." In: *Medical Image Analysis* 42 (2017), pp. 60–88. ISSN: 1361-8415. DOI: https://doi.org/10.1016/j.media.2017.07.005.

[7] B. Shen, S. Guo, C. Lyu, Y. Yan, D. Zhao, Z. Lin, B. Wang, and M. Ding. "Generating Synthetic X-ray Image Methods for Vascular Interventional Surgery." In: *2024 IEEE International Conference on Mechatronics and Automation (ICMA)*. IEEE. 2024, pp. 715–720.

[8] P. Ambrosini, D. Ruijters, W. J. Niessen, A. Moelker, and T. van Walsum. "Fully Automatic and Real-Time Catheter Segmentation in X-Ray Fluoroscopy." In: *CoRR* abs/1707.05137 (2017). arXiv: 1707.05137.

[9] K. K. Kumamaru, B. E. Hoppel, R. T. Mather, and F. J. Rybicki. "CT Angiography: Current Technology and Clinical Use." In: *Radiologic Clinics of North America* 48.2 (2010). CT Angiography, pp. 213–235. ISSN: 0033-8389. DOI: https://doi.org/10.1016/j.rcl.2010.02.006.

[10] M. Unberath, J.-N. Zaech, S. C. Lee, B. Bier, J. Fotouhi, M. Armand, and N. Navab. *DeepDRR – A Catalyst for Machine Learning in Fluoroscopy-guided Procedures*. 2018. arXiv: 1803.08606 [physics.med-ph].

[11] S. Jaganathan, M. Kukla, J. Wang, K. Shetty, and A. Maier. "Self-supervised 2D/3D registration for X-ray to CT image fusion." In: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*. 2023, pp. 2788–2798.

[12] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel. "Domain randomization for transferring deep neural networks from simulation to the real world." In: *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2017, pp. 23–30. DOI: 10.1109/IROS.2017.8202133.

[13] Y. Nagai and Y. Kouno. "Characteristics of X-ray Fluoroscopy." In: *Respiratory Endoscopy*. Ed. by T. Izumo, S. Sasada, T. Aso, K. Nasu, and Y. Arai. Singapore: Springer Singapore, 2017, pp. 13–16. ISBN: 978-981-287-916-5. DOI: 10.1007/978-981-287-916-5_3.

[14] A. K. Jones, S. Balter, P. Rauch, and L. K. Wagner. "Medical imaging using ionizing radiation: optimization of dose and image quality in fluoroscopy." In: *Medical physics* 41.1 (2014), p. 014301.

[15] M. Kipling, A. Mohammed, and R. Medding. "Guidewires in clinical practice: Applications and troubleshooting." In: *Expert review of medical devices* 6 (Apr. 2009), pp. 187–95. DOI: 10.1586/17434440.6.2.187.

[16] B. Zhang, S. Faghihroohi, M. F. Azampour, S. Liu, R. Ghotbi, H. Schunkert, and N. Navab. "A Patient-Specific Self-supervised Model for Automatic X-Ray/CT Registration." In: *Medical Image Computing and Computer Assisted Intervention – MICCAI 2023*. Ed. by H. Greenspan, A. Madabhushi, P. Mousavi, S. Salcudean, J. Duncan, T. Syeda-Mahmood, and R. Taylor. Cham: Springer Nature Switzerland, 2023, pp. 515–524. ISBN: 978-3-031-43996-4.

[17] A. Klöckner, N. Pinto, Y. Lee, B. Catanzaro, P. Ivanov, and A. Fasih. "PyCUDA and PyOpenCL: A scripting-based approach to GPU run-time code generation." In: *Parallel computing* 38.3 (2012), pp. 157–174.

[18] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. "Pytorch: An imperative style, high-performance deep learning library." In: *Advances in neural information processing systems* 32 (2019).

[19] R. Varghese and S. M. "YOLOv8: A Novel Object Detection Algorithm with Enhanced Performance and Robustness." In: *2024 International Conference on Advances in Data Engineering and Intelligent Computing Systems (ADICS)*. 2024, pp. 1–6. DOI: 10.1109/ADICS58448.2024.10533619.

[20] G. Jocher, J. Qiu, and A. Chaurasia. *Ultralytics YOLO*. Version 8.0.0. Jan. 2023.

[21] J. Redmon, S. K. Divvala, R. B. Girshick, and A. Farhadi. "You Only Look Once: Unified, Real-Time Object Detection." In: *CoRR* abs/1506.02640 (2015). arXiv: 1506.02640.

[22] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky. "BoT-SORT: Robust associations multi-pedestrian tracking." In: *arXiv preprint arXiv:2206.14651* (2022).

[23] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft. "Simple online and realtime tracking." In: *2016 IEEE international conference on image processing (ICIP)*. Ieee. 2016, pp. 3464–3468.

[24] T. Fischer, T. E. Huang, J. Pang, L. Qiu, H. Chen, T. Darrell, and F. Yu. "Qdtrack: Quasi-dense similarity learning for appearance-only multiple object tracking." In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45.12 (2023), pp. 15380–15393.

[25] Y. Zhang, S. Miao, T. Mansi, and R. Liao. "Task Driven Generative Modeling for Unsupervised Domain Adaptation: Application to X-ray Image Segmentation." In: *CoRR* abs/1806.07201 (2018). arXiv: 1806.07201.

[26] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros. "Unpaired image-to-image translation using cycle-consistent adversarial networks." In: *Proceedings of the IEEE international conference on computer vision*. 2017, pp. 2223–2232.

[27] E. Arazo, D. Ortego, P. Albert, N. E. O'Connor, and K. McGuinness. "Pseudo-labeling and confirmation bias in deep semi-supervised learning." In: *2020 International joint conference on neural networks (IJCNN)*. IEEE. 2020, pp. 1–8.

[28] Y. Li, L. Guo, and Y. Ge. "Pseudo labels for unsupervised domain adaptation: A review." In: *Electronics* 12.15 (2023), p. 3325.

[29] R. Gharleghi, D. Adikari, K. Ellenberger, S.-Y. Ooi, C. Ellis, C.-M. Chen, R. Gao, Y. He, R. Hussain, C.-Y. Lee, J. Li, J. Ma, Z. Nie, B. Oliveira, Y. Qi, Y. Skandarani, J. L. Vilaça, X. Wang, S. Yang, A. Sowmya, and S. Beier. "Automated segmentation of normal and diseased coronary arteries – The ASOCA challenge." In: *Computerized Medical Imaging and Graphics* 97 (2022), p. 102049. ISSN: 0895-6111. DOI: https://doi.org/10.1016/j.compmedimag.2022.102049.

[30] Tzutalin. *LabelImg*. Free software: MIT License. 2015.

[31] J. Wasserthal, H.-C. Breit, M. T. Meyer, M. Pradella, D. Hinck, A. W. Sauter, T. Heye, D. T. Boll, J. Cyriac, S. Yang, M. Bach, and M. Segeroth. "TotalSegmentator: Robust Segmentation of 104 Anatomic Structures in CT Images." In: *Radiology: Artificial Intelligence* 5.5 (2023), e230024. DOI: 10.1148/ryai.230024. eprint: https://doi.org/10.1148/ryai.230024.

[32] R. Beare, B. Lowekamp, and Z. Yaniv. "Image segmentation, registration and characterization in R with SimpleITK." In: *Journal of statistical software* 86 (2018), pp. 1–35.

[33] B. C. Lowekamp, D. T. Chen, L. Ibáñez, and D. Blezek. "The design of SimpleITK." In: *Frontiers in neuroinformatics* 7 (2013), p. 45.

[34] Z. Yaniv, B. C. Lowekamp, H. J. Johnson, and R. Beare. "SimpleITK image-analysis notebooks: a collaborative environment for education and reproducible research." In: *Journal of digital imaging* 31.3 (2018), pp. 290–303.

[35] D. Atkinson. "Geometry in medical imaging: DICOM and NIfTI formats." In: (2022).

[36] R. Kikinis, S. D. Pieper, and K. G. Vosburgh. "3D Slicer: A Platform for Subject-Specific Image Analysis, Visualization, and Clinical Support." In: *Intraoperative Imaging and Image-Guided Therapy*. Ed. by F. A. Jolesz. New York, NY: Springer New York, 2014, pp. 277–289. ISBN: 978-1-4614-7657-3. DOI: 10.1007/978-1-4614-7657-3_19.

[37] L. Antiga, M. Piccinelli, L. Botti, B. Ene-Iordache, A. Remuzzi, and D. A. Steinman. "An image-based modeling framework for patient-specific computational hemodynamics." In: *Medical & Biological Engineering & Computing* 46.11 (2008), pp. 1097–1112. ISSN: 1741-0444. DOI: 10.1007/s11517-008-0420-1.

[38] A. Azizi, C. Tremblay, and S. Martel. "Trajectory planning for vascular navigation from 3D angiography images and vessel centerline data." In: *2017 International Conference on Manipulation, Automation and Robotics at Small Scales (MARSS)*. IEEE. 2017, pp. 1–6.

[39] R. Izzo, D. Steinman, S. Manini, and L. Antiga. "The vascular modeling toolkit: a python library for the analysis of tubular structures in medical images." In: *Journal of Open Source Software* 3.25 (2018), p. 745.

[40] A. Hagberg, P. J. Swart, and D. A. Schult. *Exploring network structure, dynamics, and function using NetworkX*. Tech. rep. Los Alamos National Laboratory (LANL), Los Alamos, NM (United States), 2008.

[41] E. Chaplin and G. Culpan. "Fluoroscopy, contrast agents and image-guided intervention." In: *Fundamentals of Diagnostic Imaging: An Introduction for Nurses and Allied Health Care Professionals* (2008), p. 22.

[42] J. Maintz and M. A. Viergever. "A survey of medical image registration." In: *Medical Image Analysis* 2.1 (1998), pp. 1–36. ISSN: 1361-8415. DOI: https://doi.org/10.1016/S1361-8415(01)80026-8.

[43] C. T. Metz, S. Klein, M. Schaap, T. van Walsum, and W. J. Niessen. "Nonrigid registration of dynamic medical imaging data using nD+ t B-splines and a groupwise optimization approach." In: *Medical image analysis* 15.2 (2011), pp. 238–249.

[44] J. Shackelford, N. Kandasamy, and G. Sharp. "Deformable Volumetric Registration Using B-Splines." In: *GPU Computing Gems Emerald Edition*. Elsevier, 2011, pp. 751–770.

[45] S. Klein, M. Staring, K. Murphy, M. A. Viergever, and J. P. W. Pluim. "elastix: A Toolbox for Intensity-Based Medical Image Registration." In: *IEEE Transactions on Medical Imaging* 29.1 (2010), pp. 196–205. DOI: 10.1109/TMI.2009.2035616.

[46] D. P. Shamonin, E. E. Bron, B. P. Lelieveldt, M. Smits, S. Klein, and M. Staring. "Fast Parallel Image Registration on CPU and GPU for Diagnostic Classification of Alzheimers Disease." In: *Frontiers in Neuroinformatics* Volume 7 - 2013 (2014). ISSN: 1662-5196. DOI: 10.3389/fninf.2013.00050.

[47] J. Ashburner and K. J. Friston. "Rigid body registration." In: *Statistical parametric mapping: The analysis of functional brain images* (2007), pp. 49–62.

[48] Z. Wu, T. Lan, J. Wang, Y. Ding, and Z. Qin. "Medical image registration using b-spline transform." In: *Int. J. Simul. Syst. Sci. Technol* 17.48 (2016), pp. 1–1.

[49] J. L. Bentley. "Multidimensional binary search trees used for associative searching." In: *Communications of the ACM* 18.9 (1975), pp. 509–517.

[50] A. Khoynezhad and M. J. Kruse. "Guidewires, catheters, and sheaths used for thoracic endografting procedures." In: *Journal of cardiac surgery* 24.2 (2009), pp. 113–119.

[51] G. G. Tóth, M. Yamane, and G. R. Heyndrickx. "How to select a guidewire: technical features and key characteristics." In: *Heart* 101.8 (2015), pp. 645–652.

[52] S. Rajaraman, G. Zamzmi, F. Yang, Z. Liang, Z. Xue, and S. Antani. "Semantically redundant training data removal and deep model classification performance: A study with chest X-rays." In: *Computerized Medical Imaging and Graphics* 115 (2024), p. 102379.

[53] J. Yoon and M.-K. Choi. "Exploring video frame redundancies for efficient data sampling and annotation in instance segmentation." In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2023, pp. 3308–3317.

[54] Z. Guan, Z. Wang, G. Zhang, L. Li, M. Zhang, Z. Shi, and N. Jiang. "Multi-object tracking review: retrospective and emerging trend." In: *Artificial Intelligence Review* 58.8 (2025), p. 235.

[55] M. Hussain. "Yolov5, yolov8 and yolov10: The go-to detectors for real-time vision." In: *arXiv preprint arXiv:2407.02988* (2024).

[56] G. Bradski. "The OpenCV Library." In: *Dr. Dobb's Journal of Software Tools* (2000).

[57] T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick, and P. Dollár. *Microsoft COCO: Common Objects in Context*. 2015. arXiv: `1405.0312 [cs.CV]`.

[58] Ultralytics. *Ultralytics Documentation*. Web page. Comprehensive online documentation of Ultralytics tools. 2025.

[59] I. Loshchilov and F. Hutter. "Decoupled weight decay regularization." In: *arXiv preprint arXiv:1711.05101* (2017).

[60] MMTracking Contributors. *OpenMMLab Video Perception Toolbox and Benchmark*. Jan. 2021.

[61] J. Cao, J. Pang, X. Weng, R. Khirodkar, and K. Kitani. "Observation-centric sort: Rethinking sort for robust multi-object tracking." In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2023, pp. 9686–9696.

[62] P. Bergmann, T. Meinhardt, and L. Leal-Taixe. "Tracking without bells and whistles." In: *Proceedings of the IEEE/CVF international conference on computer vision*. 2019, pp. 941–951.

[63] L. Leal-Taixé, A. Milan, I. Reid, S. Roth, and K. Schindler. "Motchallenge 2015: Towards a benchmark for multi-target tracking." In: *arXiv preprint arXiv:1504.01942* (2015).

[64] S. Ren, K. He, R. Girshick, and J. Sun. "Faster R-CNN: Towards real-time object detection with region proposal networks." In: *IEEE transactions on pattern analysis and machine intelligence* 39.6 (2016), pp. 1137–1149.

[65] K. He, X. Zhang, S. Ren, and J. Sun. "Deep residual learning for image recognition." In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.

[66] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie. "Feature pyramid networks for object detection." In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, pp. 2117–2125.

[67] X. Liu, C. Yoo, F. Xing, H. Oh, G. El Fakhri, J.-W. Kang, J. Woo, et al. "Deep unsupervised domain adaptation: A review of recent advances and perspectives." In: *APSIPA Transactions on Signal and Information Processing* 11.1 (2022).

[68] S. Jecklin, Y. Shen, A. Gout, D. Suter, L. Calvet, L. Zingg, J. Straub, N. A. Cavalcanti, M. Farshad, P. Fürnstahl, et al. "Domain adaptation strategies for 3D reconstruction of the lumbar spine using real fluoroscopy data." In: *Medical Image Analysis* 98 (2024), p. 103322.

[69] Y. Yang and S. Soatto. "Fda: Fourier domain adaptation for semantic segmentation." In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2020, pp. 4085–4095.

[70] J. Kay, T. Haucke, S. Stathatos, S. Deng, E. Young, P. Perona, S. Beery, and G. Van Horn. "Align and distill: Unifying and improving domain adaptive object detection." In: *arXiv preprint arXiv:2403.12029* (2024).

[71] G. Mattolin, L. Zanella, E. Ricci, and Y. Wang. "Confmix: Unsupervised domain adaptation for object detection via confidence-based mixing." In: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*. 2023, pp. 423–433.

[72] A. H. Jonathon Luiten. *TrackEval*. `https://github.com/JonathonLuiten/TrackEval`. 2020.

[73] J. Luiten, A. Osep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixé, and B. Leibe. "Hota: A higher order metric for evaluating multi-object tracking." In: *International journal of computer vision* 129.2 (2021), pp. 548–578.

[74] K. Bernardin and R. Stiefelhagen. "Evaluating multiple object tracking performance: the clear mot metrics." In: *EURASIP Journal on Image and Video Processing* 2008.1 (2008), p. 246309.

[75] E. Ristani, F. Solera, R. Zou, R. Cucchiara, and C. Tomasi. "Performance measures and a data set for multi-target, multi-camera tracking." In: *European conference on computer vision*. Springer. 2016, pp. 17–35.